

CS7140 Lecture Notes on Statistical Learning and Its Applications (Spring 2026)

Hongyang R. Zhang, Assistant Professor
Khoury College of Computer Sciences, Northeastern University

March 16, 2026

Contents

1	Overview	3
1.1	What is this course about? (Lecture 1)	3
1.2	Supervised prediction (Lecture 1)	4
1.2.1	Empirical risk minimization	5
1.2.2	Uniform convergence and generalization gap	5
1.3	Multi-layer neural networks and generative models (Lecture 2)	6
1.4	Transfer learning and minimax estimation (Lecture 2)	7
1.4.1	Transfer learning setup	7
1.4.2	Transfer learning estimators	8
1.4.3	Optimality of the estimator	9
2	Uniform convergence in supervised prediction	10
2.1	Learning a realizable, finite hypothesis class (Lecture 3)	11
2.2	Concentration estimates (Lecture 4)	12
2.2.1	Moment generating functions	14
2.2.2	Chernoff bounds for the sum of Poisson trials	15
2.3	Using uniform convergence to reason about generalization (Lecture 5)	17
2.4	Rademacher complexity (Lecture 6)	18
2.4.1	Motivation	18
2.4.2	Definition of Rademacher complexity	19
2.4.3	Generalization bounds	20
2.4.4	Properties of Rademacher complexity	21
2.4.5	Proof of Rademacher complexity generalization bounds (optional)	21
2.5	Examples of using Rademacher complexity (Lectures 7-8)	25
2.5.1	Learning finite hypothesis classes	25
2.5.2	Norm-constrained linear models	26
2.5.3	Learning two-layer neural networks	27
2.6	Matrix completion (Lecture 9)	29
2.6.1	Recovery guarantees	30

3	Optimization and generalization in neural networks	32
3.1	Neural tangent kernels (Lectures 10-12)	32
3.1.1	Motivation	33
3.1.2	Definition	34
3.1.3	Convergence analysis (optional)	36
3.2	Implicit regularization (Lectures 13)	39
3.2.1	Error dynamics (optional)	41
3.2.2	Signal dynamics (optional)	43
3.3	Benign overfitting (Lecture 13)	44
3.4	PAC-Bayes generalization bounds (Lectures 14)	45
3.5	Noise-sensitivity generalization bounds (Lecture 15)	50
4	Statistical modeling of representation learning	53
4.1	Transfer learning linear regression (Lecture 15)	53
4.1.1	Bias and variance	54
4.1.2	Numerical example	55
4.1.3	Minimax lower bounds	55
4.2	Multitask and meta-learning (Lecture 15)	57
5	Optimization and generalization in reinforcement learning	58
5.1	Value iteration and policy iteration (Lecture 16)	60
5.2	Sample complexity and online learning (Lecture 16)	61

1 Overview

Machine learning has been increasingly used in technology platforms and products, affecting our daily lives. Machine learning involves a collection of models, algorithms, and engineering frameworks:

- Regression and classification: least squares estimation, logistic regression, ℓ_1/ℓ_2 -regularization, bias-variance tradeoff, cross-validation.
- Neural networks and deep learning: convolutional neural networks, backpropagation, foundation models, language modeling.
- Unsupervised learning: dimension reduction such as principal component analysis, clustering, contrastive learning.
- Reinforcement learning and sequential decision-making: robotics, reinforcement learning from human feedback.
- Generative AI: large language models, diffusion models, multi-modal data.
- Causal machine learning: study the cause-and-effect to estimate the counterfactual.
- Machine learning libraries: numpy, sklearn, pytorch, tensorflow, huggingface...

1.1 What is this course about? (Lecture 1)

This course aims to uncover the common **mathematical and statistical principles** underlying the diverse array of machine learning models and algorithms. This class primarily focuses on the theoretical analysis of learning algorithms and models. Many of the techniques introduced in this course—which involve a beautiful blend of probability, linear algebra, and optimization—are separate fields in their respective discipline with independent interests outside of machine learning. For example, we will study the supremum of a complex random variable corresponding to the outcome of a learning algorithm applied to train a neural network model. We will show how to design estimation algorithms when working under distribution shifts between the training and test datasets.

From a practical point of view, studying the underlying working mechanisms of a learning algorithm can deepen our understanding of how machine learning models work. For example, suppose we want to design a neural network classifier to predict the sentiment of a document. We train a regression model using word frequencies as features and achieve 100% training accuracy on 1000 training documents and 85% test accuracy on 1000 test documents. How can we reduce the gap between training and test accuracy? Further, what happens if the word frequencies between the training corpus and the test corpus are different? It is possible to answer these questions from an engineering perspective; instead, this course will mostly focus on the mathematical analysis underlying these procedures, although we will provide computational examples from time to time to help you understand the theoretical concepts.

There is a clear gap between theoretical analysis and an algorithm's practical performance. For instance, theoretical analysis is usually conducted under standard technical assumptions that are often made to simplify the analysis. The goal, instead, is to *build a deeper understanding of the underlying key concepts through mathematical modeling and theoretical analysis*. We will see if we succeed in accomplishing this objective by the end of this semester.

The course materials are divided into three parts: *fundamental concepts of statistical learning theory* (January), *generalization and optimization of neural networks and deep learning* (February), and *statistical modeling of emerging learning paradigms* (March).¹

1.2 Supervised prediction (Lecture 1)

Central questions: *Does minimizing training error lead to low test error? How does the generalization ability depend on the model architecture and the training algorithm?* It turns out that answering these questions is highly non-trivial as it also depends on the underlying data distribution.

To formally study these questions, let us first describe the mathematical setup:

- Let $\mathcal{X}$ denote the feature space. Let $\mathcal{Y}$ denote the space of all possible outcomes. Binary classification example: $\mathcal{X} = \mathbb{R}^d$, $\mathcal{Y} = \{+1, -1\}$
- Consider the problem of predicting an output $y \in \mathcal{Y}$ given an input $x \in \mathcal{X}$.
- Let $\mathcal{H}$ be a set of hypotheses. Linear model example:

$$\mathcal{H} = \left\{ x \mapsto \beta^\top x + \epsilon : \forall \beta \in \mathbb{R}^d, \epsilon \in \mathbb{R} \right\}$$

- Let $\ell : (\mathcal{X}, \mathcal{Y}) \times \mathcal{H} \rightarrow \mathbb{R}$ be a loss function. For example, the mean squared error (MSE) applied to linear models is

$$\ell((x, y), \epsilon) = \left(\beta^\top x + \epsilon - y \right)^2, \forall \beta \in \mathbb{R}^d, \forall \epsilon \in \mathbb{R}$$

- Given n training data samples, denoted by $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, the training loss (or empirical risk) of a hypothesis $h \in \mathcal{H}$ is defined as

$$\hat{L}(h) = \frac{1}{n} \sum_{i=1}^n \ell(h(x_i), y_i), \forall h \in \mathcal{H} \quad (1)$$

We make a critical assumption about the data-generating process. We assume that every x_i, y_i pair is drawn independently and identically from an unknown distribution $\mathbb{P}^*$, supported on $\mathcal{X} \times \mathcal{Y}$.

The test loss (or expected risk) of a hypothesis $h \in \mathcal{H}$ is then given by

$$L(h) = \mathbb{E}_{(x, y) \sim \mathbb{P}^*} [\ell(h(x), y)]. \quad (2)$$

Example 1.1 (Linear regression). *To make the above setup more concrete, perhaps the best example would be linear regression. There are many standard texts on this topic; see, e.g., Wainwright, 2019. In a standard parametric regression setup, we have n samples $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, where every $x_i \in \mathbb{R}^p$ is a p -dimensional feature vector drawn from some unknown distribution $\mathcal{D}$, and $y_i \in \mathbb{R}$, for every $i = 1, 2, \dots, n$. In addition, suppose that there exists an unknown $\beta \in \mathbb{R}^p$ such that*

$$y_i = x_i^\top \beta + \varepsilon_i, \text{ for every } i = 1, 2, \dots, n, \quad (3)$$

¹April will be dedicated to course project presentations.

where $x_i^\top \beta = \sum_{j=1}^p x_{i,j} \beta_j$, and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ is a noise random variable with mean zero and variance σ^2 .

Given n samples, the goal of this problem is to learn a linear model parameterized by $\hat{\beta}$ that achieves the lowest mean-squared error (MSE) on an unseen sample.

Remark 1.2. We have assumed the training and test distributions are the same. While this assumption does not hold exactly in practice, morally, the training and test distributions must be related.

Formulating what it means to be related and not related, and addressing the discrepancy between training and test data, are studied in the area of domain adaptation or transfer learning.

The independence assumption, which also does not hold exactly in practice, ensures that more training data gives us more information.

1.2.1 Empirical risk minimization

Consider minimizing the training loss

$$\hat{h}_{\text{ERM}} \leftarrow \underset{h \in \mathcal{H}}{\operatorname{argmin}} \hat{L}(h). \quad (4)$$

What can say that the relationship between $\hat{L}(\hat{h}_{\text{ERM}})$ and $L(\hat{h}_{\text{ERM}})$? A key challenge is that the randomness of $\hat{h}_{\text{ERM}}$ now depends on $\hat{L}$. Thus, $\hat{L}(\hat{h}_{\text{ERM}})$ involves a correlation between the training data samples and the minimizing hypothesis. A central aspect we will tackle in the first part of the course is developing the machinery to address this challenge.

Example 1.3 (Pretraining and fine-tuning). *An emerging learning paradigm that has emerged over the past few years follows a two-stage procedure involving pretraining on a large amount of unlabeled data, followed by fine-tuning on a small amount of labeled data.*

The pretraining stage usually follows some masked prediction procedure on unlabeled data. The supervised fine-tuning (SFT) procedure can be formulated with the above ERM setup.

- Suppose we have some model like a neural network, f_{W_0} , parameterized by some initialization W_0 .
- There is a small amount of training dataset, S , from which we compute the training loss $\hat{L}(f_{W_0})$.
- SFT corresponds to minimizing $\hat{L}(f_{W_0})$, usually via a stochastic gradient optimization algorithm.
- An important consideration in SFT is overfitting, since the model is pretrained on a large amount of unlabeled data. The size of the model is usually much larger than the size of the training dataset S .

1.2.2 Uniform convergence and generalization gap

In the first part of this course, we will show various “uniform convergence” statements of the following flavor:

With probability at least $1 - \delta$, the gap between test loss and training loss of any hypothesis is upper bounded by some small ϵ , that is, $L(h) - \hat{L}(h) \leq \epsilon$, where the ϵ is generally a function that depends on δ and other aspects of the learning algorithm/model

More rigorously, we would like to show statements of the following:

$$\Pr \left[\underbrace{L(h) - \hat{L}(h)}_{\text{Generalization gap}} > \epsilon \right] \leq 1 - \delta, \quad (5)$$

where the randomness is on the training data samples drawn from $\mathbb{P}^*$. This statement essentially quantifies the **generalization gap** between the training and test losses of the machine learning model.

Equipped with such a statement, we will then apply the statement to the empirical risk minimizer $\hat{h}_{\text{ERM}}$, since the result essentially holds for any $h \in \mathcal{H}$, which also subsumes $\hat{h}_{\text{ERM}}$ as a special case.

1.3 Multi-layer neural networks and generative models (Lecture 2)

Consider the case of a basic one-layer network:

$$f_{a,W,b}(x) := x \rightarrow \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \text{ where} \quad (6)$$

- σ is the nonlinear activation function. Typical choices of σ : ReLU $x \rightarrow \max(0, x)$, sigmoid $x \rightarrow \frac{1}{1+\exp(-x)}$. Key property: Lipschitz-continuity: A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is said to be *C-Lipschitz-continuous* if the following is true:

$$|f(x) - f(y)| \leq C \cdot |x - y|.$$

- $Z = \{\alpha = (a_i, w_i, b_i)\}_{i=1}^m$ are trainable parameters of the network. By varying α , we define the function class $\mathcal{H}$ as

$$\mathcal{H} = \{f_\alpha : \forall \alpha \in Z\}.$$

- $\mathcal{H}$ essentially represents a set of one-hidden-layer neural networks with m neurons.
- Let $W = [w_1, w_2, \dots, w_m]$, and $b = [b_1, b_2, \dots, b_m]$. We may write $f_{a,W,b}$ equation (6) as $x \rightarrow a^\top \sigma(Wx + b)$.

By extending the above setup, we may write a deep network as

$$f_\alpha(x) = \sigma_L(W_L \sigma_{L-1}(\dots \sigma_2(W_2 \sigma_1(W_1 x + b_1) + b_2) \dots)), \quad (7)$$

where α now encodes all the parameters of the network. The depth of the network is given by L . The width is given by $\max(m_1, m_2, \dots, m_L)$, i.e., the layer with the most neurons in the layer.

Motivating questions: *How could we analyze the training and test losses of a deep network? How well does a deep network generalize, and how does it depend on its depth and width?*

How does this ability to learn and to generalize rely on the data distributions, and what is the role of optimization algorithms used to train the network?

A *language model* specifies a conditional probability distribution $\Pr_\theta(\cdot \mid P)$, given a prompt sequence P , produces the next-token according to underlying probability masses.

Example 1.4 (In-context learning). *To illustrate the concept of a language model, let us consider a few-shot meta-learning problem (Garg et al., 2022). In this problem, each prompt P_{θ_i} involves a sequence of examples or demonstrations*

$$P_{\theta_i} := (x_1, y_1, x_2, y_2, \dots, x_{t-1}, y_{t-1}, x_t),$$

ended with a query example x_t . The goal is to predict the correct output y_t corresponding to the query x_t .

To make this more concrete, suppose that $y_j = \theta_i^\top x_j$, for every $j = 1, 2, \dots, t-1$. The desired output $y_t = \theta_i^\top x_t$.

- *At training time, the model sees a sequence of prompt-answer pairs (P_{θ_i}, y_t) .*
- *At test time, the model sees a new prompt P_θ parameterized by some unknown θ . The model is asked to first “solve” the linear regression from the in-context examples given in P_θ , and then use the “learned” regression model to output the correct answer corresponding to the query.*

1.4 Transfer learning and minimax estimation (Lecture 2)

An important learning paradigm that has emerged in the past few years is transfer learning—transferring the knowledge from one task to help solve another task. How could we develop a more rigorous statistical modeling of transfer learning? A better understanding of this question has applications in language modeling, computer vision, robotics, to name a few.

1.4.1 Transfer learning setup

Perhaps the simplest modeling framework is to examine transfer learning in linear regression tasks. For example, we may consider the case of two linear regression tasks, one called the source task and the other called the target task.

Suppose we have n_1 samples from the source task. We have n_2 samples from the target task. How could we use the samples from the source task to help estimate the target task? Concretely, let the samples of the source task be denoted by $(x_1^{(1)}, y_1^{(1)}), (x_2^{(1)}, y_2^{(1)}), \dots, (x_{n_1}^{(1)}, y_{n_1}^{(1)})$, where every $x_i^{(1)}$ is a p -dimensional vector and $y_i^{(1)}$ is a real-valued outcome. Similarly, we denote the samples of the target task as $(x_1^{(2)}, y_1^{(2)}), (x_2^{(2)}, y_2^{(2)}), \dots, (x_{n_2}^{(2)}, y_{n_2}^{(2)})$.

Now we can ask a few more concrete questions:

- How does the difference between the $x \rightarrow y$ mappings affect transfer learning performance?
- How does the covariance between the feature vectors of source and target tasks affect transfer learning?

More generally, we may say that the source task and the target task involve a distribution shift between their them. In the area of domain adaptation (Kouw and Loog, 2018):

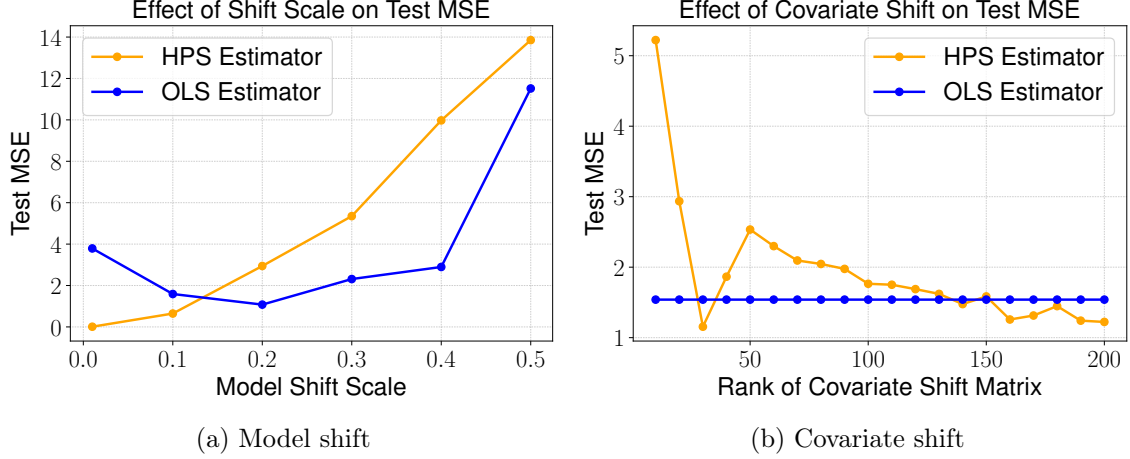


Figure 1: Illustrating the effects of model shift and covariate shift in transfer learning linear regression.

- Covariate shift refers to scenarios where both tasks follow the same model conditioned on the features, but they have different feature distributions.
- Model shift refers to scenarios where the two tasks follow different models conditioned on the same features.

We may now ask, how does covariate shift and model shift affect transfer learning performance?

1.4.2 Transfer learning estimators

Typically, there are two strategies for transfer learning, one called hard transfer, where we hard-code the shared component across tasks, the other called soft transfer, where we use separate components for task, and encourage the separate components to be close to each other (Ruder, 2017; Dhifallah and Lu, 2021).

Example 1.5 (Illustration of model and covariate shifts in linear regression). *We shall assume that the source task follows a linear relation specified by an unknown parameter $\beta^{(1)} \in \mathbb{R}^p$:*

$$y_i^{(1)} = x_i^{(1)\top} \beta^{(1)} + \epsilon_i^{(1)}, \text{ for all } i = 1, 2, \dots, n_1, \quad (8)$$

where $\epsilon_i^{(1)}$ is a white noise with mean zero and variance σ_1^2 .

We further assume that the target task follow another linear relation specified by an unknown parameter $\beta^{(2)} \in \mathbb{R}^p$:

$$y_i^{(2)} = x_i^{(2)\top} \beta^{(2)} + \epsilon_i^{(2)}, \text{ for all } i = 1, 2, \dots, n_2, \quad (9)$$

where $\epsilon_i^{(2)}$ is a white noise with mean zero and variance σ_2^2 .

In the context of linear regression, we can define an hard parameter sharing estimator as follows:

$$\hat{L}^{\text{HPS}}(\beta) = \frac{1}{n_1 + n_2} \left(\sum_{i=1}^{n_1} \left(x_i^{(1)\top} \beta - y_i^{(1)} \right)^2 + \sum_{j=1}^{n_2} \left(x_j^{(2)\top} \beta - y_j^{(2)} \right)^2 \right) \quad (10)$$

We may also elect to use a soft parameter sharing estimator instead:

$$\hat{L}^{\text{SPS}}(\beta, z) = \frac{1}{n_1 + n_2} \left(\sum_{i=1}^{n_1} \left(x_i^{(1)\top} (\beta + z) - y_i^{(1)} \right)^2 + \sum_{j=1}^{n_2} \left(x_j^{(2)\top} \beta - y_j^{(2)} \right)^2 \right) + \lambda \|z\|^2 \quad (11)$$

Essentially, by adjusting λ , we can adjust the magnitude of z , which then determines how far (and how close) the source and target task models are.

A natural baseline is when we do not use the source task data at all. That is, perform least squares regression using target task data alone.

In Figure 1, we illustrate the effects incurred from model shifts and covariate shifts in transfer learning linear regression, comparing between HPS and OLS. In particular, we capture model shift as the distance error between $\beta^{(1)}$ and $\beta^{(2)}$, and we capture covariate shift as the difference in the population covariance matrix between task one and task two. To generate the condition matrix, we use a rank- r matrix whose trace is equal to p , and set its nonzero eigenvalues as p/r .

1.4.3 Optimality of the estimator

The above estimation algorithms are based on the best practices of practitioner (see the surveys above). Suppose we analyze their performances. However, how can we know that there are no better estimators out there? How could we understand the fundamental limits of estimation and optimization procedures? These are often called *minimax lower bounds* on the performance of estimators, and it usually falls into the area of information theory (Duchi, 2019). In particular, we will touch on the framework of minimax lower bounds for transfer learning (though the scope of this is much broader than we'll cover in our lectures).

2 Uniform convergence in supervised prediction

Recall that we have introduced the empirical loss and expected loss of a hypothesis (denoted by $L(h)$ and $\hat{L}(h)$) for some h in a hypothesis class $\mathcal{H}$. Suppose we minimize the empirical risk to get $\hat{h}_{\text{ERM}}$. Two questions:

- Generalization gap: how does the expected and empirical risks compare for ERM, i.e., $L(\hat{h}_{\text{ERM}}) - \hat{L}(\hat{h}_{\text{ERM}})$? This is called the **generalization gap**.
- Excess risk: how well does ERM do with respect to the best possible hypothesis in the hypothesis class, i.e., $L(\hat{h}_{\text{ERM}}) - \min_{h \in \mathcal{H}} L(h)$? This is also called the **excess risk**.

A particularly fruitful framework for analyzing learning algorithms is the probably approximately correct (PAC) framework (Valiant, 1984):

A learning algorithm A PAC learns a hypothesis class $\mathcal{H}$ if

- For any distribution $\mathbb{P}^*$ supported over $\mathcal{X} \times \mathcal{Y}$, and any $\epsilon > 0$, $\delta > 0$
- Upon taking n I.I.D. samples from $\mathbb{P}^*$, A produces an output $\hat{h} \in \mathcal{H}$ such that with probability at least $1 - \delta$ (over the randomness of the samples)

$$L(\hat{h}) - \min_{h \in \mathcal{H}} L(h) \leq \epsilon$$

- Further, n is a polynomial function of $\epsilon^{-1}, \delta^{-1}, d, |\mathcal{H}|$, and A runs in time polynomial in $n, d, \epsilon^{-1}, \delta^{-1}$ (where d is the dimension of the input)

Remark: Notice that the running time complexity places a bound on the sample complexity as well. We will assume that the empirical risk minimizer can be computed efficiently. For instance, think of a large neural network whose training loss can be efficiently reduced to reach zero using stochastic gradient descent.

Example 2.1 (Policy learning). *Even though the above definition was proposed three decades ago, it remains a very fundamental concept, and applies broadly beyond supervised prediction.*

Consider a finite Markov decision process (MDP) with state space $\mathcal{S}$, action space $\mathcal{A}$, horizon H , and unknown transition and reward dynamics. Let Π be a finite class of deterministic policies $\pi : \mathcal{S} \rightarrow \mathcal{A}$. For any policy $\pi \in \Pi$, we can define the expected loss as

$$L(\pi) := \mathbb{E} \left[\sum_{t=1}^H \ell(s_t, a_t) \right],$$

where the trajectory (s_t, a_t) is generated by executing π in the MDP, and ℓ is the loss measured at each step.

Once we collect n trajectories (e.g., via an exploration policy),² we can write down the empirical loss averaged over the n sampled trajectories. Therefore, a policy learning algorithm

²For instance, a uniform random exploration picks an action $a \in \mathcal{A}$ uniformly at random at any state. This guarantees unbiased coverage of all actions. An ϵ -greedy exploration instead follows a fixed deterministic policy with probability $1 - \epsilon$, while choosing a random action with probability ϵ .

A PAC-learns the policy space Π if A can produce a near-optimal policy with excess loss at most ϵ with probability at least $1 - \delta$, and n only depends polynomially on $|\mathcal{S}|, |\mathcal{A}|, \delta^{-1}$.

2.1 Learning a realizable, finite hypothesis class (Lecture 3)

Next, we give a concrete example to illustrate learnability in a finite, realizable hypothesis class.

Assumptions (realizable, finite hypothesis): i) The size of the hypothesis space, $\mathcal{H}$, is finite; ii) There exists a hypothesis $h^* \in \mathcal{H}$ such that h^* achieves perfect performance, i.e.,

$$L(h^*) = \mathbb{E}_{(x,y) \in \mathbb{P}^*} [\ell(h^*(x), y)] = 0.$$

Under these assumptions, we shall prove the following property of ERM:

Under the above assumptions, with probability $1 - \delta$,

$$L(\hat{h}_{\text{ERM}}) \leq \frac{\log(|\mathcal{H}|) + \log(\delta^{-1})}{n} \quad (12)$$

In particular, to reduce the expected risk below ϵ , we want

$$n \geq \frac{\log(|\mathcal{H}|) + \log(\delta^{-1})}{\epsilon}.$$

Example 2.2. Imagine a clinic designing a digital diagnostic tool to determine if a patient should be referred to a specialist based on three binary (Yes/No) symptoms: 1. Fever? 2. Cough? 3. Shortness of breath?

Since there are only 3 binary features, there are only 8 possible patient profiles. A hypothesis in this setup is a rule that assigns a “Refer” or “Not Refer” label to each profile. The number of possible ways to assign binary labels to these 8 profiles is $2^8 = 256$, which is finite. A hypothesis is any rule that assigns a binary labeling to the 8 profiles. Thus, the size of the hypothesis space is 256 in total.

If a medical board has already decided a specific protocol regarding the referral, then there exists a true target function in the 256 options. Thus, suppose you train the model on historical data from a clinic, you are guaranteed that at least one hypothesis in your class can achieve zero error, because the data was generated by a specific rule.

Proof. We’d like to upper bound the probability of the bad event that $L(\hat{h}_{\text{ERM}}) > \epsilon$: Let $B \subseteq \mathcal{H}$ be the set of bad hypotheses: $\{h \in \mathcal{H} : L(h) > \epsilon\}$

We can rewrite our goal as upper bounding the probability of selecting a bad hypothesis $\Pr[L(\hat{h}_{\text{ERM}}) > \epsilon] = \Pr[\hat{h}_{\text{ERM}} \in B]$. Recall the empirical risk of ERM is always zero because at least $\hat{L}(h^*) = 0$. Hence for any “bad” hypothesis in B , they must have zero empirical risk

$$\Pr[\hat{h}_{\text{ERM}} \in B] \leq \Pr[\exists h \in B : \hat{L}(h) = 0]$$

Now we shall deal with the above in two steps. First, bound $\Pr[\hat{L}(h) = 0]$ for a fixed $h \in B$. Notice that on a random example from $\mathcal{P}^*$, the accuracy of h should be $1 - L(h)$. Since the training data is drawn IID from $\mathcal{P}^*$, and $L(h) \geq \epsilon$ for any $h \in B$, we get that

$$\Pr[\hat{L}(h) = 0] \leq (1 - L(h))^n \leq (1 - \epsilon)^n \leq \exp^{-\epsilon n},$$

where we use the fact that $1 - x \leq \exp(-x)$.

Second, we want the above to hold simultaneously for all $h \in B$. We can apply the union bound to bound the probability of all bad events:

$$\begin{aligned} \Pr[\exists h \in B : \hat{L}(h) = 0] &\leq \sum_{h \in B} \Pr[\hat{L}(h) = 0] \\ &\leq |B| \exp(-\epsilon n) \\ &\leq |\mathcal{H}| \exp(-\epsilon n) \end{aligned}$$

By setting the above at most δ , we conclude that ϵ must be at least $\frac{\log(|\mathcal{H}|) + \log(\delta^{-1})}{n}$. This concludes the proof for learning finite, realizable hypothesis spaces.

Two remarks:

- The excess risk only grows logarithmically with the size of the hypothesis class, so we can afford to use an exponentially large hypothesis space.
- The result is independent of $\mathbb{P}^*$. This is nice because typically we don't know the true distribution.

The proof of this result is elementary but illustrates an important pattern that will recur in more complex scenarios. We are interested in the expected risk, but only have access to empirical risk to choose the ERM:

- Step 1 (convergence): for a fixed h , show that $\hat{L}(h)$ is close to $L(h)$ with high probability.
- Step 2 (uniform convergence): show that the above holds simultaneously for all hypotheses $h \in \mathcal{H}$.

However, the assumptions are restrictive. There exists a perfect hypothesis (realizability). What happens when the problem is not realizable? To answer this, we introduce the tools of concentration estimates.

Second, the hypothesis class is finite. What happens when the number of hypotheses is infinite? To answer this, we need better ways of measuring the “size” of a set – leading to Rademacher complexity, VC dimension, and PAC-Bayes bounds (to name a few).

2.2 Concentration estimates (Lecture 4)

Concentration inequalities are powerful tools from probability theory that show the average of independent random variables will be concentrated around its expectation. Concentration estimates are the basis of a large branch of learning theory (Bach, 2024) and high-dimensional statistics (Wainwright, 2019). They are one of the most basic tools for studying supervised learning algorithms and models (Zhang, 2023), primarily because much of supervised learning deals with in-distribution samples.

Example 2.3 (Mean estimation). *Let $X_1, X_2, \dots, X_n$ be independent and identically distributed random variables with mean $\mu = \mathbb{E}[X_i]$, for all $i = 1, 2, \dots, n$. Define the empirical mean as*

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n X_i$$

How does $\hat{\mu}_n - \mu$ behave? There are at least three types of statements one could make:

- **Consistency:** by law of large numbers, $\hat{\mu}_n - \mu \rightarrow 0$.
- **Asymptotic normality:** let the variance of X_i (for all i) be equal to σ^2 , by the central limit theorem, we have $\sqrt{n}(\hat{\mu}_n - \mu) \sim \mathcal{N}(0, \sigma^2)$.
- **Tail estimates:** for our purpose, we would like to draw a statement of the following type $\Pr[|\hat{\mu}_n - \mu| \geq \epsilon] \leq \delta$. For getting such tail estimates, we typically need to study the tail of a distribution, for instance, the tail of a Gaussian distribution, etc.

Markov's inequality: Let $Z \geq 0$ be a non-negative random variable. Then

$$\Pr[Z \geq t] \leq \frac{\mathbb{E}[Z]}{t}$$

Proof. Since Z is a non-negative quantity, we always have the condition that

$$t\mathbb{1}_{Z \geq t} \leq Z$$

To see this, notice that if $Z \geq t$, then the above is true. On the other hand, if $Z < t$, then the left-hand side above is zero, whereas $Z \geq 0$. Next, take the expectation over Z on both sides, we get

$$\mathbb{E}[t\mathbb{1}_{Z \geq t}] \leq \mathbb{E}[Z] \Rightarrow \mathbb{E}[\mathbb{1}_{Z \geq t}] \leq \frac{\mathbb{E}[Z]}{t}$$

Notice that $\mathbb{E}[\mathbb{1}_{Z \geq t}] = \Pr[Z \geq t]$. Thus, we have shown that Markov's inequality is true.

Chebyshev's inequality: Let X be a random variable with mean equal to μ . Then

$$\Pr[|X - \mu| \geq \epsilon] \leq \frac{\text{Var}[X]}{\epsilon^2}$$

Proof. We will use Markov's inequality to get this result. Let $Z = (X - \mu)^2$ and let $t = \epsilon^2$. Notice that $Z \geq 0$. Thus, based on Markov's inequality

$$\Pr[Z \geq t] \leq \frac{\mathbb{E}[Z]}{t} = \frac{\mathbb{E}[(X - \mu)^2]}{t} = \frac{\text{Var}[X]}{t},$$

which completes the proof.

Hoeffding's inequality: Let $Z_1, Z_2, \dots, Z_n$ be n independent and identically distributed random variables drawn from a distribution with expectation μ and whose values are bounded from above by one.

Let $\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n Z_i$ denote the mean of the n random variables. Then, for any $\epsilon \in (0, 1)$, we have

$$\Pr[|\hat{\mu}_n - \mu| > \epsilon] \leq 2 \exp(-2\epsilon^2 n). \quad (13)$$

The Hoeffding's inequality is a very powerful result when we work with the average of n random variables. Variants of this inequality (which is restricted to bounded random variables) are also called Chernoff bound.³ Next, we shall see a proof through the use of moment generating functions (MGF).

³https://en.wikipedia.org/wiki/Chernoff_bound

2.2.1 Moment generating functions

Definition 2.4 (Moment generating functions). *For a random variable X , its MGF is given by*

$$M_X(t) := \mathbb{E}[\exp(tX)]$$

One can also think of the MGF in terms of Taylor's expansion of $\exp(tX)$ as

$$M_X(t) = 1 + t \mathbb{E}[X] + \frac{t^2}{2} \mathbb{E}[X^2] + \frac{t^3}{6} \mathbb{E}[X^3] + \dots$$

Thus, the first moment is the mean of X . The second moment is the variance of X (assuming the mean of X is zero). And so on.

The MGF of the sum of two independent random variables X and Y is the product of the MGF of X and Y , respectively.

- To see this, notice that

$$M_{X+Y}(t) = \mathbb{E}[e^{t(X+Y)}] = \mathbb{E}[e^{tX}e^{tY}] = \mathbb{E}[e^{tX}] \mathbb{E}[e^{tY}] = M_X(t)M_Y(t)$$

- Here we have used that X and Y are independent, and hence e^{tX} and e^{tY} are independent, to conclude that $\mathbb{E}[e^{tX}e^{tY}] = \mathbb{E}[e^{tX}] \mathbb{E}[e^{tY}]$

Example 2.5 (Gaussian random variables). *In this example, we shall work through the above Definition 2.4 in the case of a Gaussian random variable. Let $X \sim \mathcal{N}(0, \sigma^2)$. Then, we will show that $M_X(t) = e^{\sigma^2 t^2 / 2}$. To derive this, we use the definition of the probability density of Gaussian random variables:*

$$\begin{aligned} M_X(t) &= \mathbb{E}[e^{tX}] = \int \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2 - 2\sigma^2 tx}{2\sigma^2}\right) dx \\ &= \int \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \sigma^2 t)^2 - \sigma^4 t^2}{2\sigma^2}\right) dx \\ &= \exp\left(\frac{\sigma^2 t^2}{2}\right) \end{aligned}$$

The high-level idea for showing the Hoeffding's inequality is obtained by applying Markov's inequality to e^{tX} for some well-chosen value t . From Markov's inequality, we can derive the following useful inequality: for any $t > 0$,

$$\Pr[X \geq a] = \Pr[e^{tX} \geq e^{ta}] \leq \frac{\mathbb{E}[e^{tX}]}{e^{ta}}$$

In particular,

$$\Pr[X \geq a] \leq \min_{t>0} \frac{\mathbb{E}[e^{tX}]}{e^{ta}} \tag{14}$$

Similarly, for any $t < 0$,

$$\Pr[X \leq a] = \Pr[e^{tX} \geq e^{ta}] \leq \min_{t<0} \frac{\mathbb{E}[e^{tX}]}{e^{ta}}$$

Bounds for specific distributions are obtained by choosing appropriate values for t . See Chapter 4 in Mitzenmacher and Upfal, 2017 for further references.

2.2.2 Chernoff bounds for the sum of Poisson trials

We now develop the most commonly used version of the Chernoff bound for the tail distribution of a sum of independent 0-1 random variables, which are also known as Poisson trials.⁴

Let $X_1, \dots, X_n$ be a sequence of independent Poisson trials with $\Pr[X_i = 1] = p_i$. Let $X = \sum_{i=1}^n X_i$, and let

$$\mu = \mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i] = \sum_{i=1}^n p_i$$

For a given $\delta > 0$, we are interested in bounds on $\Pr[X \geq (1 + \delta)\mu]$ and $\Pr[X \leq (1 - \delta)\mu]$, that is, the probability that X deviates from its expectation μ by $\delta\mu$ or more. To develop a Chernoff bound, we need to compute the moment generating function of X . We start with the MGF of each X_i :

$$\begin{aligned} M_{X_i}(t) &= \mathbb{E}[e^{tX_i}] = p_i e^t + (1 - p_i) = 1 + p_i(e^t - 1) \\ &\leq e^{p_i(e^t - 1)}, \end{aligned}$$

where in the last step, we have used the fact that, for any y , $1 + y \leq e^y$. Since the X_i 's are independent from each other, we take the product of the n MGF to obtain

$$\begin{aligned} M_X(t) &= \prod_{i=1}^n M_{X_i}(t) \leq \prod_{i=1}^n e^{p_i(e^t - 1)} \\ &= \exp\left(\sum_{i=1}^n p_i(e^t - 1)\right) = e^{(e^t - 1)\mu} \end{aligned}$$

Based on this result, we now apply Markov's inequality: for any $t > 0$, we have

$$\begin{aligned} \Pr[X \geq (1 + \delta)\mu] &= \Pr[e^{tX} \geq e^{t(1+\delta)\mu}] \\ &\leq \frac{\mathbb{E}[e^{tX}]}{e^{t(1+\delta)\mu}} \\ &\leq \frac{e^{(e^t - 1)\mu}}{e^{t(1+\delta)\mu}} \end{aligned}$$

For any $\delta > 0$, we can set $t = \ln(1 + \delta) > 0$ to get

$$\Pr[X \geq (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}}\right)^\mu$$

For $0 < \delta \leq 1$, we need to show that

$$\frac{e^\delta}{(1 + \delta)^{1+\delta}} \leq e^{-\delta^2/3}$$

Taking the logarithm of both sides, we obtain

$$f(\delta) = \delta - (1 + \delta) \ln(1 + \delta) + \frac{\delta^2}{3} \leq 0$$

⁴Poisson trials differ from Poisson random variables.

Computing the derivatives of $f(\delta)$, we have:

$$f'(\delta) = 1 - \frac{1+\delta}{1+\delta} - \ln(1+\delta) + \frac{2\delta}{3}, f''(\delta) = -\frac{1}{1+\delta} + \frac{2}{3}$$

We see that the second derivative is less than zero if $\delta < 1/2$, and it is positive otherwise. Hence, $f'(\delta)$ first decreases and then increases in the interval $[0, 1]$. Since $f'(0) = 0$ and $f'(1) < 0$, we conclude that $f'(\delta) \leq 0$ in the interval $[0, 1]$. Since $f(0) = 0$, it follows that $f(\delta) \leq 0$, which shows that for any δ between 0 and 1, we have

$$\Pr[X \geq (1+\delta)\mu] \leq e^{-\mu\delta^2/3}$$

Next, let us look at an example of Chernoff bounds.

Example 2.6 (Coin flips). *Let X be the number of heads in a sequence of n independent fair coin flips. That is $X = X_1 + X_2 + \dots + X_n$, where every X_i is a Bernoulli random variable yielding one with probability $1/2$. Applying the Chernoff bound, we have*

$$\Pr\left[\left|X - \frac{n}{2}\right| \geq \frac{1}{2}\sqrt{6n \ln n}\right] \leq 2 \exp\left(-\frac{1}{3} \frac{n}{2} \frac{6 \ln n}{n}\right) = \frac{2}{n}$$

*This demonstrates that the concentration of the number of heads around the mean $n/2$ is very tight. Most of the time, the deviations from the mean are on the order of $O(\sqrt{n \ln n})$.*⁵

Example 2.7 (Gaussian random variables). *Recall that the MGF of a standard Gaussian random variable is $M_X(t) = e^{\sigma^2 t^2/2}$. We derive a tail bound by plugging in $M_X(t)$ to equation (14):*

$$\Pr[X \geq \epsilon] \leq \inf_t \exp\left(\frac{\sigma^2 t^2}{2} - t\epsilon\right)$$

The minimum of the RHS is attained by setting $t = \frac{\epsilon}{\sigma^2}$, yielding:

$$\Pr[X \geq \epsilon] \leq \exp\left(-\frac{\epsilon^2}{2\sigma^2}\right)$$

What about non-Gaussian random variables? Notice that the bound still holds if we replace $M_X(t)$ with an upper bound. This motivates the following definition.

Definition 2.8 (Sub-Gaussian). *A mean zero random variable X is **sub-Gaussian** with parameter σ^2 if its MGF is bounded as follows:*

$$M_X(t) \leq \exp\left(\frac{\sigma^2 t^2}{2}\right) \tag{15}$$

It follows that for sub-Gaussian X , we again have that $\Pr[X \geq \epsilon] \leq \exp(-\epsilon^2/(2\sigma^2))$.

⁵Another use case of Chernoff bound: Suppose we are interested in evaluating the probability that a particular gene mutation occurs in the population. Given a DNA sample, a lab test can determine if it carries the mutation. However, the lab test is expensive and we would like to obtain a relatively reliable estimate from a small number of tests.

Example 2.9 (Revisiting ERM in the lens of sub-Gaussian random variables). *Imagine you are training a classifier. You want to know if the training error is a good proxy for the test error. For most ML loss functions, their loss values are bounded (e.g., between 0 and 1 or another small constant $C > 0$). Next, we shall verify that any bounded random variable is sub-Gaussian. For any $t > 0$, suppose that X is a bounded random variable between a and b . First, the variance of X is at most $(b - a)^2/4$. To see this,*

$$\begin{aligned} \text{Var}[X] &= \inf_t \mathbb{E}[(X - t)^2] && \text{(because the minimum is achieved when } t = \mathbb{E}[X]) \\ &\leq \mathbb{E}\left[\left(X - \frac{b+a}{2}\right)^2\right] && \text{(one can verify that } \mathbb{E}[(t - \frac{b+a}{2})(\frac{b+a}{2} - X)] \leq 0) \\ &\leq \max\left((b - (b+a)/2)^2, (a - (b+a)/2)^2\right), \end{aligned}$$

which is $(b - a)^2/4$.

Next, let $\varphi(\lambda) = \log \mathbb{E}_P[e^{\lambda X}]$. Let $Q_\lambda(x)$ be the distribution of x defined by

$$dQ_\lambda(x) = \frac{e^{\lambda x}}{\mathbb{E}_{X \sim P}[e^{\lambda X}]} dP(x)$$

Next, let us look at $\varphi'(\lambda)$ and $\varphi''(\lambda)$.

$$\begin{aligned} \varphi'(\lambda) &= \frac{\mathbb{E}_P[X e^{\lambda X}]}{\mathbb{E}_P[e^{\lambda X}]} = \mathbb{E}_{Q_\lambda}[X], \\ \varphi''(\lambda) &= \frac{\mathbb{E}_P[X^2 e^{\lambda X}]}{\mathbb{E}_P[e^{\lambda X}]} - \frac{(\mathbb{E}_P[X e^{\lambda X}])^2}{(\mathbb{E}_P[e^{\lambda X}])^2} = \text{Var}_{Q_\lambda}[X] \leq \frac{(b - a)^2}{4} \end{aligned}$$

Lastly, by Taylor's theorem, there exists λ' between 0 and λ (when $\lambda > 0$; The case when $\lambda < 0$ is similar) such that

$$\varphi(\lambda) = \varphi(0) + \varphi'(0)\lambda + \frac{1}{2}\varphi''(\lambda')\lambda^2 \leq \frac{\lambda^2(b - a)^2}{8}$$

Plugging this back to the expression of $\varphi(\lambda)$ completes the proof.

2.3 Using uniform convergence to reason about generalization (Lecture 5)

We now give a high-level picture of the logic behind how we can use uniform convergence to reason about generalization in the context of ERM. This will lead us to the concept of Rademacher complexity, which is a crucial notion in statistical learning theory. We would like to show that the excess risk of ERM is small:

$$\Pr\left[L(\hat{h}_{\text{ERM}}) - L(h^*) \geq \epsilon\right] \leq \delta \quad (16)$$

We can expand the excess risk as

$$L(\hat{h}) - L(h^*) = \underbrace{\left(L(\hat{h}) - \hat{L}(\hat{h})\right)}_{\text{Uniform convergence}} + \underbrace{\left(\hat{L}(\hat{h}) - \hat{L}(h^*)\right)}_{\leq 0} + \underbrace{\left(\hat{L}(h^*) - L(h^*)\right)}_{\text{Concentration}} \quad (17)$$

We'll see how concentration estimates can be used to control this difference in the third part. However, the same reasoning does not apply to the first part because the ERM $\hat{h}_{\text{ERM}}$ depends on the training examples $\hat{L}$. In particular,

$$\hat{L}(\hat{h}_{\text{ERM}}) = \frac{1}{n} \sum_{i=1}^n \ell(\hat{h}_{\text{ERM}}(x_i), y_i). \quad (18)$$

Example 2.10 (Stochastic optimization). *In practice, $\hat{h}_{\text{ERM}}$ is often computed using stochastic optimization, such as stochastic gradient descent (SGD), which updates:*

$$h^{(t+1)} = h^{(t)} - \eta_t \nabla \ell(h^{(t)}(x_{i_t}, y_{i_t})),$$

where i_t is sampled uniformly from $1, 2, \dots, n$.

Each SGD step is an unbiased estimate of the gradient of $\hat{L}(h)$, but the final iterate $\hat{h}_{\text{ERM}}$ depends on the entire dataset through the optimization trajectory.

Due to the correlation, $\hat{L}$ is not a sum of independent random variables. The central thesis of uniform convergence is that if we could ensure that $L(h)$ and $\hat{L}(h)$ are close for all $h \in \mathcal{H}$, then $L(\hat{h}_{\text{ERM}})$ must be close to $\hat{L}(\hat{h}_{\text{ERM}})$ as well. In summary, the goal of uniform convergence can be stated as

$$\Pr[L(\hat{h}_{\text{ERM}}) - L(h^*) \geq \epsilon] \leq \Pr\left[\left(\sup_{h \in \mathcal{H}} |L(h) - \hat{L}(h)|\right) \geq \frac{\epsilon}{2}\right] \leq \delta \quad (19)$$

In particular, the $1/2$ above comes from combining the error terms from the first and third parts together. Put it in words, we'd like to upper bound the probability that the largest difference between the empirical risk and the expected risk is larger than $\epsilon/2$.

2.4 Rademacher complexity (Lecture 6)

Both of our generalization bounds require finite hypothesis classes. What about infinite hypothesis classes? Note that we cannot directly apply the union bound to infinite hypothesis classes. Instead, we need a more sophisticated way to measure complexity of a hypothesis class. With Rademacher complexity, the key idea is **symmetrization**. Along the way, we need an extension of the Hoeffding's inequality to some function of bounded random variables, which is known as McDiarmid's inequality. Let us first start by motivating why we need these tools.

2.4.1 Motivation

Recall that, within the uniform convergence framework, we want to get a statement of the following

$$\Pr[L(\hat{h}) - L(h^*) \geq \epsilon] \leq \Pr\left[\sup_{h \in \mathcal{H}} |L(h) - \hat{L}(h)| \geq \frac{\epsilon}{2}\right] \leq \delta \quad (20)$$

Since there are two cases here, let us denote

$$G_n := \sup_{h \in \mathcal{H}} L(h) - \hat{L}(h), \quad G'_n := \sup_{h \in \mathcal{H}} \hat{L}(h) - L(h)$$

Then we have that

$$\Pr\left[\sup_{h \in \mathcal{H}} |L(h) - \hat{L}(h)| \geq \frac{\epsilon}{2}\right] \leq \Pr\left[G_n \geq \frac{\epsilon}{2}\right] + \Pr\left[G'_n \geq \frac{\epsilon}{2}\right]$$

Let us focus on the first case, since the second case will be similar.

Now, our main object becomes G_n : This is a rather non-trivial function, because of taking the supremum over a sum of random variables. Instead, we will look at its expectation (or first moment).

2.4.2 Definition of Rademacher complexity

Our main object of interest is now

$$\mathbb{E}[G_n] = \mathbb{E} \left[\sup_{h \in \mathcal{H}} (L(h) - \hat{L}(h)) \right].$$

Recall that $\hat{L}(h)$ is the empirical risk, and $L(h)$ is the expected risk.

This quantity is still quite difficult because it depends on the expected risk, an expectation over the unknown data distribution. The key idea of symmetrization is to remove this expected risk term with a simpler term. Let us imagine n data points $Z'_1 = (x'_1, y'_1), Z'_2 = (x'_2, y'_2), \dots, Z'_n = (x'_n, y'_n)$, sampled from the true data distribution. Then, clearly $L(h) = \mathbb{E}[\hat{L}'(h)]$, where $\hat{L}'(h)$ is the empirical risk on this copy dataset. Hence,

$$\begin{aligned} \mathbb{E}[G_n] &= \mathbb{E}_{Z_{1:n}} \left[\sup_{h \in \mathcal{H}} (L(h) - \hat{L}(h)) \right] \\ &= \mathbb{E}_{Z_{1:n}} \left[\sup_{h \in \mathcal{H}} \mathbb{E}_{Z'_{1:n}} [\hat{L}'(h) - \hat{L}(h)] \right] \\ &= \mathbb{E}_{Z_{1:n}} \left[\mathbb{E}_{Z'_{1:n}} \left[\sup_{h \in \mathcal{H}} (\hat{L}'(h) - \hat{L}(h)) \right] \right] \end{aligned}$$

Let us remove the dependence on the copy dataset. To that end, we introduce independent Rademacher random variables $\sigma_1, \sigma_2, \dots, \sigma_n$ sampled uniformly from $\{+1, -1\}$. Notice that

$$\hat{L}'(h) - \hat{L}(h) = \sum_{i=1}^n (\ell(h(x'_i), y'_i) - \ell(h(x_i), y_i)),$$

is symmetric around zero. Hence, multiplying each individual term by σ_i does not change its distribution. Thus,

$$\begin{aligned} \mathbb{E}[G_n] &\leq \mathbb{E}_{Z_{1:n}, Z'_{1:n}} \left[\sup_{h \in \mathcal{H}} (\hat{L}'(h) - \hat{L}(h)) \right] \\ &= \mathbb{E}_{Z_{1:n}, Z'_{1:n}, \sigma_{1:n}} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i (\ell(h(x'_i), y'_i) - \ell(h(x_i), y_i)) \right] \\ &\leq 2 \mathbb{E}_{Z_{1:n}, \sigma_{1:n}} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i \ell(h(x_i), y_i) \right] \end{aligned} \tag{21}$$

The last line (without the factor of 2) is the **Rademacher complexity** of $\mathcal{H}$.

In summary, let $\mathcal{H}$ be a hypothesis class consisting of a class of real-valued functions. Example: two-layer neural nets, linear models. Define the Rademacher complexity (or Rademacher average) of $\mathcal{H}$ to be

$$R_n(\mathcal{H}) := \mathbb{E}_{Z_{1:n}, \sigma_{1:n}} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i \ell(h(x_i), y_i) \right],$$

where $Z_i = (x_i, y_i)$ is a random sample from the underlying data distribution, and σ_i is sampled uniformly from $\{+1, -1\}$.

Example 2.11 (Rademacher complexity of ℓ_2 -norm bounded linear predictors). Let $\mathcal{H} = \{z \rightarrow \langle w, z \rangle \mid w \in \mathcal{W}\}$ be a set of linear functions such that $\|w\| \leq B$ for any $w \in \mathcal{W}$. Furthermore, assume the data distribution has a bounded second moment:

$$\mathbb{E}_{z \sim \mathbb{P}} [\|z\|^2] \leq C^2$$

Then we have

$$R_n(\mathcal{H}) = \mathbb{E} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i h(z_i) \right] \leq \frac{B \cdot C}{\sqrt{n}}$$

Example 2.12 (Rademacher complexity of two-layer neural networks with bounded layer norms). Consider a two-layer ReLU neural net with m hidden units. Let $\Phi = (w \in \mathbb{R}^m, U \in \mathbb{R}^{m \times d})$ denote the parameters for the first and second layers, respectively.

Given an input vector $x \in \mathbb{R}^d$, let

$$f_\Phi(x) = w^\top \phi(Ux) \in \mathbb{R},$$

denote the output of the network, where $\phi(\cdot)$ is the activation function that is 1-Lipschitz continuous. Let us assume the training set $\{x_i\}_{i=1}^n \subseteq \mathbb{R}^d$ lies in some bounded ℓ_2 -norm ball: $\|x_i\| \leq 1$ for all $i = 1, 2, \dots, n$.

Let $\mathcal{H} = \{f_\Phi \mid \|w\| \leq B', \|U\|_2 \leq B\}$. Then the Rademacher complexity of $\mathcal{H}$ is bounded by

$$R_n(\mathcal{H}) \leq 2BB' \sqrt{\frac{m}{n}}$$

Imagine we use a two-layer neural network to classify MNIST digits. If we use a network with m neurons. This result says that the Rademacher complexity scales with the operator norm of the first layer weights and the second layer weights.

2.4.3 Generalization bounds

To see the power of the concept we just introduced, here is a very general statement where we can always rely on when we work with supervised learning algorithms and models. Define

$$\mathcal{A} = \{(x, y) \rightarrow \ell(h(x), y) : h \in \mathcal{H}\}$$

as the loss function composed with the hypothesis space. Let $\mathcal{R}_n(\mathcal{A})$ denote the Rademacher complexity of the function class $\mathcal{A}$. With probability at least $1 - \delta$,

$$L(\hat{h}_{\text{ERM}}) - L(h^*) \leq 4R_n(\mathcal{A}) + \sqrt{\frac{2 \log(2\delta^{-1})}{n}} \quad (22)$$

Recall that $\hat{h}_{\text{ERM}}$ is the empirical risk minimizer (ERM), h^* is the expected risk minimizer, and n is the size of the training set.

Example 2.13 (Interpreting G_n for linear predictors). Let $\mathcal{H} = \{x \mapsto \langle w, x \rangle : \|w\|_2 \leq B\}$ and assume the loss $\ell(h(x), y)$ is bounded. For a fixed hypothesis h , the quantity $L(h) - \hat{L}(h)$ measures how much the empirical risk underestimates the true risk. The supremum in G_n searches for the worst-case hypothesis whose empirical performance looks overly optimistic. Geometrically, G_n quantifies how well the training data fits noise. If there exists a direction w such that the correlation between w and data align on the sample but not in expectation,

then G_n will be large. Using symmetrization, we showed that $\mathbb{E}[G_n] \leq 2R_n(\mathcal{A})$, where $\mathcal{A} = (x, y) \mapsto \ell(h(x), y) \in \mathcal{H}$. For linear predictors with bounded norm, this implies $\mathbb{E}[G_n] \lesssim \frac{BC}{\sqrt{n}}$, where $C^2 = \mathbb{E}[\|x\|^2]$. Thus, as the sample size grows, even the most overfit hypothesis cannot exploit randomness in the data beyond $O(n^{-1/2})$ scaling.

2.4.4 Properties of Rademacher complexity

Boundedness:

$$R_n(\mathcal{A}) \leq \max_{h \in \mathcal{H}} \max_{x, y} \ell(h(x), y)$$

This only shows that the Rademacher complexity is bounded by some constant. Usually, we'd like to show it goes to zero as n goes to infinity.

Monotonicity: If $\mathcal{H}_1 \subseteq \mathcal{H}_2$, then $R_n(\mathcal{H}_1) \leq R_n(\mathcal{H}_2)$.

This is because we now take the supreme over a larger set, hence R_n increases.

Scaling: $R_n(c \cdot \mathcal{H}) = c \cdot R_n(\mathcal{H})$. Multiplying the functions within $\mathcal{H}$ by a fixed constant $c > 0$ increases the Rademacher complexity of $\mathcal{H}$ by a factor of c .

Lipschitz composition: Suppose ϕ is a Lipschitz-continuous function, bounded by some constant c_ϕ . Recall a function is Lipschitz-continuous if changing x only changes the function value by c_ϕ times. In addition, $\phi(0) = 0$.

Let $\phi \circ \mathcal{H} = \{(x, y) \rightarrow \phi(h(x), y) : h \in \mathcal{H}\}$, i.e., compose ϕ with h . Then, we have that

$$R_n(\phi \circ \mathcal{H}) \leq c_\phi \cdot R_n(\mathcal{H}).$$

The proof essentially uses the contraction property of the sum of Rademacher random variables. For details, see Corollary 3.17 of Ledoux and Talagrand, 2013. This property is useful because we can start by studying a simpler hypothesis class, and then compose more functions with the class without going through the calculation again.

Convex hull: The convex hull of a set $\mathcal{H}$ is when we take all possible linear combinations of the hypotheses in $\mathcal{H}$. One useful property is that taking the convex of $\mathcal{H}$ preserves the Rademacher complexity: $R_n(\mathcal{H}) = R_n(\text{convex-hull}(\mathcal{H}))$.

To see this, notice that the supreme over the convex hull must be attained at a vertex of the convex hull. This property is quite useful if we want to calculate the Rademacher complexity of a polytope. Though it is an infinite set, it suffices to examine the vertices of the polytope. We'll use this property when we examine the ℓ_1 -regularization.

2.4.5 Proof of Rademacher complexity generalization bounds (optional)

The proof involves three steps:

- Show that empirical Rademacher complexity is close to the expectation.
- Use McDiarmid's inequality, which is essentially a concentration result for functions of independent random variables.

- Show that the Rademacher complexity upper bounds the excess risk. We've seen this step from Section 2.4.1.

In more detail, recall that

$$G_n = \sup_{h \in \mathcal{H}} L(h) - \hat{L}(h).$$

Our first claim is to show that G_n is close to $\mathbb{E}[G_n]$:

$$\Pr[G_n \geq \mathbb{E}[G_n] + \epsilon] \leq \exp(-2n\epsilon^2) \quad (23)$$

This shows that G_n is indeed close to its expectation plus a small error. Hence it suffices to upper bound its expectation.

Our second claim is to show that $\mathbb{E}[G_n]$ is upper bounded by the Rademacher complexity.

$$\mathbb{E}[G_n] \leq 2R_n(\mathcal{A}).$$

We have already seen the proof of this claim. Combined together, we can derive (22).

Recall that (23) that we'd like to show G_n and its expectation $\mathbb{E}[G_n]$ are close to each other. In order to complete this result, we'll use a tool called McDiarmid's inequality.

McDiarmid's inequality (The bounded differences inequality): Let f be a function satisfying the following bounded differences condition for all $i = 1, 2, \dots, n$ and all $x_1, x_2, \dots, x_n$, and any x'_i :

$$|f(x_1, x_2, \dots, x_n) - f(x_1, \dots, x_{i-1}, x'_i, x_{i+1}, \dots, x_n)| \leq c_i$$

In other words, modifying one coordinate does not change the function value too much.

Let $X_1, X_2, \dots, X_n$ be independent random variables. Then

$$\Pr[|f(X_1, X_2, \dots, X_n) - \mathbb{E}[f(X_1, X_2, \dots, X_n)]| \geq \epsilon] \leq \exp\left(-\frac{2\epsilon^2}{\sum_{i=1}^n c_i^2}\right) \quad (24)$$

Remark 2.14 (McDiarmid's inequality generalizes Hoeffding's inequality). *Let the function*

$$f(x_1, x_2, \dots, x_n) = \frac{1}{n} \sum_{i=1}^n x_i$$

be the empirical mean. Suppose every x_i and x'_i are bounded from above by one. Then, changing x_i to x'_i changes the function value by at most $\frac{2}{n}$, leading to $c_i = \frac{2}{n}$ for all $i = 1, 2, \dots, n$. Hence, the sum of c_i^2 is equal to $\frac{4}{n}$. Plugging this to (24) recovers the Hoeffding's inequality.

This result is quite powerful, since it holds for any independent random variables, and f can be complex (say a neural net). As long as this function is not too sensitive to perturbations in one coordinate, we get good concentration.

Proof of equation (24). Recall the definition of a martingale sequence A sequence of random variables $Z_0, Z_1, \dots, Z_n$ is a martingale sequence with respect to another sequence of random variables $X_1, X_2, \dots, X_n$ if and only if Z_i only depends on $X_{1:i}$ and

$$\mathbb{E}[Z_i | X_{1:i-1}] = Z_{i-1}.$$

For a martingale sequence, we have the concentration because of Azuma-Hoeffding's inequality: Suppose $Z_{1:t}$ is a martingale sequence such that

$$\mathbb{E}[Z_i | Z_{<i}] = 0, \text{ and } |Z_i| \leq c_i.$$

Then with probability at least $1 - \delta$,

$$\sum_{i=1}^n Z_i \leq \sqrt{2 \left(\sum_{i=1}^n c_i^2 \right) \log(\delta^{-1})} \quad (25)$$

For reference, see Chapter 12.4 of Mitzenmacher and Upfal (2017).⁶

We're going to construct a sequence

$$Z_i = \mathbb{E}[f(X_1, X_2, \dots, X_n) | X_{1:i}] \quad (26)$$

Let's check the following properties

- Clearly, Z_i only depends on the values of $X_{1:i}$, but not of the remaining sequence
- Besides,

$$\mathbb{E}[Z_i | X_{1:i-1}] = \mathbb{E}[\mathbb{E}[f(X_1, X_2, \dots, X_n) | X_{1:i}] | X_{1:i-1}] = Z_{i-1}$$

- Note that for $i = 0$,

$$Z_0 = \mathbb{E}[f(X_1, X_2, \dots, X_n)]$$

- For $i = n$,

$$Z_n = \mathbb{E}[f(X_1, \dots, X_n) | X_{1:n}] = f(X_1, \dots, X_n)$$

Using these notations, we'd like to bound the following

$$\Pr[Z_n - Z_0 \geq \epsilon]$$

Let $D_i = Z_i - Z_{i-1}$. Clearly, $Z_n - Z_0 = \sum_{i=1}^n D_i$. We're going to show that $|D_i|$ is bounded by c_i . Therefore, using Azuma-Hoeffding's inequality, we can get an exponentially decreasing tail bound for $Z_n - Z_0$. We write down both Z_i and Z_{i-1} as follows:

$$\begin{aligned} Z_{i-1} &= \mathbb{E}[f(X_1, \dots, X_n) | X_{1:i-1}] \\ Z_i &= \mathbb{E}[f(X_1, \dots, X_n) | X_{1:i}] \end{aligned}$$

The key difference is whether we condition on X_i or not above. Imagine the set of possible realizations of X_i . We'll consider the maximum and minimum achieving realizations

$$\begin{aligned} L_i &= \inf_x \mathbb{E}[f(X_1, \dots, X_{i-1}, X_i = x, \dots, X_n) | X_{1:i-1}, X_i = x] - Z_{i-1} \\ U_i &= \sup_x \mathbb{E}[f(X_1, \dots, X_{i-1}, X_i = x, \dots, X_n) | X_{1:i-1}, X_i = x] - Z_{i-1} \end{aligned}$$

Clearly, D_i is somewhere between L_i and U_i . Suppose L_i is achieved by $X_i = x_L$ and U_i is achieved by $X_i = x_U$.

⁶See also https://en.wikipedia.org/wiki/Azuma%27s_inequality

By the bounded difference assumption, for any fixed values $X_{1:n}$ except X_i (being x_L or x_U), they are at most c_i apart. Taking expectation over $X_{1:i-1}$ and note that they are independent of X_i (key!), we have that

$$|U_i| = |\mathbb{E}[f(X_1, \dots, X_{i-1}, X_i = x_U, \dots, X_n) - f(X_1, \dots, X_{i-1}, X_i, \dots, X_n) \mid X_{1:i-1}]| \leq c_i,$$

by the bounded difference condition on X_i . And similarly, $|L_i| \leq c_i$. Thus, we have verified that the sequence $D_1, D_2, \dots, D_n$ is bounded. In addition, we can check that they are a Martingale sequence based on the properties listed above.

Having checked through McDiarmid's inequality, it remains to show that G_n satisfies the bounded difference condition. Recall that

$$G_n = \sup_{h \in \mathcal{H}} L(h) - \hat{L}(h).$$

This quantity naturally arises as we derive a uniform convergence claim. Recall that $\hat{L}$ is the empirical risk. Consider changing Z_i to Z'_i in G_n , we'd like to bound the difference

$$\left| \left(\sup_{h \in \mathcal{H}} L(h) - \hat{L}(h) \right) - \left(\sup_{h \in \mathcal{H}} \left(L(h) - \hat{L}(h) + \frac{1}{n}(\ell(Z_i, h) - \ell(Z'_i, h)) \right) \right) \right| \leq \frac{1}{n}$$

This is because the loss values are within 0 and 1. Taking supreme cannot increase the difference.

We now put all the pieces together to complete the proof of equation (22).

Proof of equation (22). We'd like to show that, for

$$\epsilon = 4R_n(\mathcal{A}) + \sqrt{\frac{2 \log(2\delta^{-1})}{n}},$$

we have

$$\Pr[L(\hat{h}_{\text{ERM}}) - L(h^*) \geq \epsilon] \leq \Pr \left[\sup_{h \in \mathcal{H}} |L(h) - \hat{L}(h)| \geq \frac{\epsilon}{2} \right] \leq \delta$$

The latter reduces to showing that

$$\Pr \left[\sup_{h \in \mathcal{H}} |L(h) - \hat{L}(h)| \geq \frac{\epsilon}{2} \right] \leq \Pr[G_n \geq \frac{\epsilon}{2}] + \Pr[G'_n \geq \frac{\epsilon}{2}]$$

By the steps above, we have

$$\begin{aligned} \Pr[G_n \geq \frac{\epsilon}{2}] &\leq \Pr \left[G_n - \mathbb{E}[G_n] \geq \sqrt{\frac{\log(2\delta^{-1})}{2n}} \right] \\ &\leq \exp \left(-2n \left(\frac{\log(2\delta^{-1})}{2n} \right) \right) = \frac{\delta}{2} \end{aligned}$$

The first line is because by equation (21),

$$\mathbb{E}[G_n] \leq 2R_n(\mathcal{A}).$$

A similar proof applies to show that $\Pr[G'_n \geq \frac{\epsilon}{2}] \leq \frac{\delta}{2}$.

2.5 Examples of using Rademacher complexity (Lectures 7-8)

So far, we have set up Rademacher complexity for bounding the complexity of an infinite hypothesis class. Now, let us apply Rademacher complexity for the case when the function class is finite. Then, we will go through several examples where the function class is infinite.

2.5.1 Learning finite hypothesis classes

Massart's finite lemma: Let $\mathcal{H}$ be a set of functions/hypotheses. Let M^2 be a bound on the second moment of functions of $\mathcal{H}$:

$$\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n (f(Z_i))^2 \leq M^2,$$

where $Z_i = (x_i, y_i)$ denotes an input data sample. Then, the empirical Rademacher complexity is bounded by:

$$\hat{R}_n(\mathcal{H}) \leq \sqrt{\frac{2M^2 \log(|\mathcal{H}|)}{n}} \quad (27)$$

Example 2.15 (Model Selection in AutoML). *In modern AutoML pipelines, we often select a model from a finite library $\mathcal{H}$ of M pre-trained architectures (e.g., different versions of Vision Transformers). Since the models are fixed and we only choose the one with the minimum empirical risk $\hat{R}_n(h)$, our hypothesis class is finite. By Massart's Lemma, the complexity of this selection process scales with $\sqrt{\log M}$, meaning we can safely choose from thousands of models without significantly increasing the generalization gap, provided our validation set n is sufficiently large. For instance, if there are 100 models, the overfitting risk from picking the best model grows only with $\sqrt{\log(100)}$.*

Proof. For simplicity, let us denote $W_f = \frac{1}{n} \sum_{i=1}^n \sigma_i f(Z_i)$. Recall that the empirical Rademacher complexity is defined as

$$\hat{R}_n(\mathcal{H}) = \mathbb{E}_{\sigma_{1:n}} \left[\sup_{f \in \mathcal{H}} W_f \mid Z_{1:n} \right]$$

Let us write down the moment generating function of $\sup_{f \in \mathcal{H}} W_f$, and we'll use the convexity of the exponential function

$$\begin{aligned} \exp \left(t \cdot \mathbb{E} \left[\sup_{f \in \mathcal{H}} W_f \mid Z_{1:n} \right] \right) &\leq \mathbb{E} \left[\exp \left(t \cdot \sup_{f \in \mathcal{H}} W_f \right) \mid Z_{1:n} \right] \\ &= \mathbb{E} \left[\sup_{f \in \mathcal{H}} \exp(t \cdot W_f) \mid Z_{1:n} \right] \end{aligned}$$

The above can be further bounded by

$$|\mathcal{H}| \cdot \mathbb{E} [\exp(t \cdot W_f) \mid Z_{1:n}]$$

Finally, recall that W_f is the sum of n independent random variables. Hence, we invoke the product property of the individual MGFs, to get that $M_{W_f}(t)$ is the product of the individual

random variables. Recall that we've conditioned on $Z_{1:n}$, which can be treated as constants. Thus, the randomness is now just on $\sigma_{1:n}$ —each σ_i is either $+1$ or -1 , scaled by $f(Z_i)$ (which is a constant conditioned on Z_i). We can show that σ_i is sub-Gaussian with parameter 1, since we have the fact that for any random variable X bounded between a and b (for some $a < b$), then X must be $(b - a)^2/4$ sub-Gaussian.⁷

As a result, W_f is sub-Gaussian with parameter at most $\frac{1}{n^2} \sum_{i=1}^n (f(Z_i))^2 \leq M^2/n$. Therefore, we now get that

$$\mathbb{E}[\exp(t \cdot W_f) \mid Z_{1:n}] \leq \exp\left(\frac{t \cdot M^2}{2n}\right)$$

We can now conclude that

$$\exp(t \hat{R}_n(f)) \leq |\mathcal{H}| \exp\left(\frac{t M^2}{2n}\right)$$

Taking log on both sides and dividing by t , we get:

$$\hat{R}_n(\mathcal{H}) \leq \frac{\log(|\mathcal{H}|)}{t} + \frac{t M^2}{2n}$$

By setting t to minimize the right-hand side above, we conclude that $\hat{R}_n(\mathcal{H}) \leq \sqrt{\frac{2 \log(|\mathcal{H}|) M^2}{n}}$.

We can apply the above result along with (22) to derive the result for learning from a bounded set of 0-1 valued functions.

2.5.2 Norm-constrained linear models

The next two examples involve learning a linear model from a norm-bounded hypothesis space. The first example applies to linear regression with ℓ_2 -norm constraints, for instance, training models with weight decay. The second example applies to linear regression with ℓ_1 -norm constraints, i.e., training with ℓ_1 -regularization. Next, we'll derive the Rademacher complexity of these two examples.

Proof of Example 2.11. The key idea is to exploit the linear algebraic structure of the Rademacher complexity

$$\begin{aligned} R_n(\mathcal{H}) &= \mathbb{E} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i h(z_i) \right] = \mathbb{E} \left[\sup_{w \in \mathcal{W}} \frac{1}{n} \sum_{i=1}^n \sigma_i \langle w, z_i \rangle \right] \\ &\leq \mathbb{E} \left[\sup_{w \in \mathcal{W}} \left\| \frac{w}{n} \right\| \cdot \left\| \sum_{i=1}^n \sigma_i z_i \right\| \right] \\ &\leq \frac{B}{n} \cdot \mathbb{E} \left[\left\| \sum_{i=1}^n \sigma_i z_i \right\| \right] \end{aligned}$$

by the condition placed on $\mathcal{W}$. Finally, by Jensen's inequality, the above is at most

$$\frac{B}{n} \cdot \sqrt{\mathbb{E} \left[\left\| \sum_{i=1}^n \sigma_i z_i \right\|^2 \right]} = \frac{B}{n} \cdot \sqrt{\sum_{i=1}^n \mathbb{E} [\|z_i\|^2]} \leq \frac{B \cdot C}{\sqrt{n}}$$

⁷For a proof see this blog post: <https://statisticaloddsandends.wordpress.com/2018/10/05/bounded-random-variables-are-sub-gaussian/>

Example 2.16 (Rademacher complexity of ℓ_1 -norm constrained balls). *In some applications, we have a finite but large set of features, and we believe that only a small subset of them are relevant to our task. For such applications, ℓ_1 -regularization has proven to be a working strategy (e.g., pruning, compression). Assume that the entries of the input vector are bounded from above by some constant $C > 0$: $\max_{j=1}^d z_{i,j} \leq C$, for $i = 1, 2, \dots, n$. Let $\mathcal{H} = \{z \rightarrow \langle w, z \rangle \mid \|w\|_1 \leq B\}$ be a set of linear functions. Then, the Rademacher complexity of $\mathcal{H}$ is bounded as follows*

$$R_n(\mathcal{H}) \leq \frac{B \cdot C \cdot \sqrt{2 \log(2d)}}{\sqrt{n}}$$

Proof. The key idea is that the ℓ_1 ball is the convex hull of $2d$ weight vectors aligned with the basis

$$W = \{Be_1, -Be_1, Be_2, -Be_2, \dots, Be_d, -Be_d\}$$

Since the Rademacher complexity of this class is equal to the Rademacher complexity of its convex hull, we just look at the finite class W since

$$R_n(\mathcal{H}) = \mathbb{E} \left[\sup_{w \in W} \frac{1}{n} \sum_{i=1}^n \sigma_i \langle w, z_i \rangle \right]$$

Since W is a finite set, we can apply Massart's finite lemma (see equation (27)). In order to do so, we shall first check the moment condition:

$$\frac{1}{n} \sum_{i=1}^n (\sigma_i \langle w, z_i \rangle)^2 \leq \|w\|_1^2 \cdot \|z\|_\infty^2 = B^2 \cdot C^2.$$

Thus, by equation (27), we now get that

$$\hat{R}_n(\mathcal{H}) \leq \sqrt{\frac{2B^2C^2 \log(2d)}{n}}.$$

2.5.3 Learning two-layer neural networks

Proof of Example 2.12. For this result, we will use composition property of Rademacher complexity and result for ℓ_2 -norm bounded linear predictors. Let us first write the Rademacher complexity as

$$R_n(\mathcal{H}) = \mathbb{E}_{\sigma_{1:n}} \left[\sup_{\Phi \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i w^\top \phi(Ux_i) \right]$$

We first deal with the supremum over the second layer weight w by setting its direction to be the same as the other vector

$$R_n(\mathcal{H}) \leq B' \cdot \mathbb{E}_{\sigma_{1:n}} \left[\sup_{\Phi \in \mathcal{H}} \left\| \frac{1}{n} \sum_{i=1}^n \sigma_i \phi(Ux_i) \right\|_2 \right]$$

Next, because there are m neurons. Hence, by Cauchy-Schwartz inequality, we reduce the above to less than

$$B' \cdot \sqrt{m} \cdot \mathbb{E} \left[\sup_{\|u\| \leq B} \left| \frac{1}{n} \sum_{i=1}^n \sigma_i \phi(u^\top x_i) \right| \right]$$

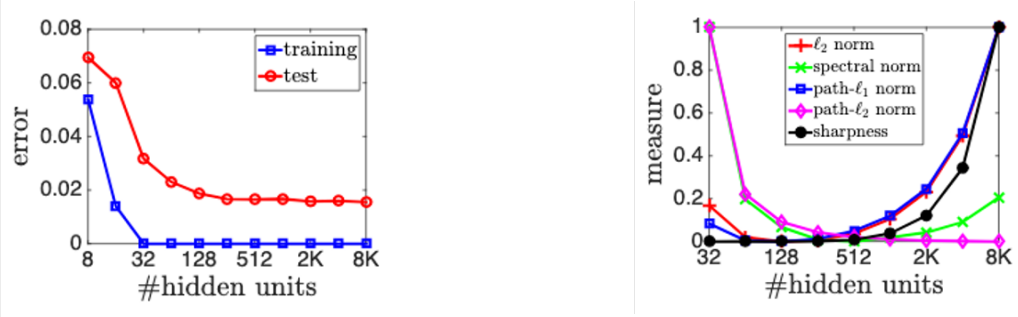


Figure 2: Vary the number of neurons m in a two-layer neural net. Path norm scales nicely with the test error. Results taken from Neyshabur et al. (2017).

The idea here is that since the operator norm of the first layer weight U is bounded by B , the norm of every row of U is also bounded by B . Furthermore, the absolute value can be bounded by adding a factor of two.

Next, we can use the composition property: ϕ is 1-Lipschitz continuous. Hence, the above is at most

$$2B' \cdot \sqrt{m} \cdot \mathbb{E} \left[\sup_{\|u\| \leq B} \frac{1}{n} \sum_{i=1}^n \sigma_i u^\top x_i \right].$$

To finish the proof, we invoke the Rademacher complexity bound for ℓ_2 -norm bounded linear predictors, to get that the above is at most $2B'B\sqrt{m/n}$.

Provided with the Rademacher complexity of two-layer neural nets, we can get a generalization bound for neural networks, with an additional error term of $\sqrt{\log(\delta^{-1})/n}$.

The above result scales with $\sqrt{m}$ – number of hidden units. Hence, the result gets worse when we over-parametrize with more hidden units. Empirically, researchers have observed that generalization error can improve as number of hidden units increases. Here's the result from training a two-layer neural net on MNIST (Neyshabur et al., 2017).

We'll show a generalization bound that does not depend on the number of hidden units m using the path norm $P(w, U) = \sum_{i=1}^m |w_i| \cdot \|u_i\|$, where u_i is the i -th row of U . Let $\mathcal{H} = \{f_\Phi \mid P(w, U) \leq B\}$. Then, the Rademacher complexity of $\mathcal{H}$ is bounded as follows

$$R_n(\mathcal{H}) \leq \frac{2B}{\sqrt{n}}.$$

Proof. Let's first write the Rademacher complexity. Recall that u_i is the i -th row of U . We expand out the Rademacher complexity using

$$w^\top \phi(Ux_i) = \sum_{j=1}^m w_j \phi(u_j^\top x_i)$$

Let $\bar{u}_j = u_j / \|u_j\|$. Because ReLU activation is 1-homogeneous, meaning that $\phi(cx) = c \cdot \phi(x)$. Hence,

$$w^\top \phi(Ux_i) = \sum_{j=1}^m w_j \|u_j\| \phi(\bar{u}_j^\top x_i)$$

We apply this expansion to the Rademacher complexity

$$R_n(\mathcal{H}) = \mathbb{E} \left[\sup_{\Phi} \frac{1}{n} \sum_{i=1}^n \sigma_i \left(\sum_{j=1}^m w_j \|u_j\| \phi(\bar{u}_j^\top x_i) \right) \right].$$

Taking the supremum over Φ means that we'll maximize the path norm above! Hence, after switching the order of the sum, we can get

$$R_n(\mathcal{H}) \leq B \cdot \mathbb{E} \left[\max_{1 \leq j \leq m} \left| \frac{1}{n} \sum_{i=1}^n \sigma_i \phi(\bar{u}_j^\top x_i) \right| \right] \leq \frac{2B}{\sqrt{n}}.$$

Remark 2.17. *The second result implies the first result. To see this, we use the Cauchy-Schwartz inequality:*

$$P(w, U) = \sum_{i=1}^m |w_i| \cdot \|u_i\| \leq \sqrt{\sum_{i=1}^m w_i^2} \cdot \sqrt{\sum_{i=1}^m \|u_i\|^2} \leq B' \cdot B \cdot \sqrt{m}$$

Technically speaking, the path norm still scales with the number of hidden units. However, as we saw from the previous slide, some neurons turn out to be quite small in the experiments, resulting in a small path norm even as m increases.

2.6 Matrix completion (Lecture 9)

Suppose you are working on a recommendation system. The goal of this system is to predict the review ratings of a movie or a music database on YouTube or on Netflix. This problem can be formulated as a matrix completion problem. Suppose there is an unknown matrix $M \in \mathbb{R}^{d_1 \times d_2}$, corresponding to the true ratings in the database. From the database, you see a subset of the entire matrix, denoted by $\widehat{M}$, such that $\widehat{M}_{i,j} = M_{i,j}$ for $(i, j) \in \Omega$ for some observed set of indices Ω , and $\widehat{M}_{i,j} = 0$ otherwise. Given the partially-observed data matrix $\widehat{M}$, how can we recover the unobserved entries outside Ω and impute the full matrix M ?

A natural, first solution is to use rank-constrained minimization. Given an input array $\widehat{M}$, we seek to find X as close to $\widehat{M}$ as possible on Ω , subject to a rank constraint:

$$\begin{aligned} \min_X \quad & \sum_{(i,j) \in \Omega} (X_{i,j} - M_{i,j})^2 \\ \text{s.t.} \quad & \text{rank}(X) = r \end{aligned}$$

However, the rank constraint leads to a non-convex problem, resulting in a search space whose complexity increases exponentially. Instead, consider relaxing the rank constraint into a nuclear norm constraint. Given a matrix $X \in \mathbb{R}^{d_1 \times d_2}$, let $\|X\|_* = \sum_{j=1}^{\min(d_1, d_2)} \lambda_j(X)$, which is the sum of all the singular values of X . One can show that $\|X\|_*$ is a convex operator in X . This is called the nuclear norm of X . Instead, consider the following nuclear norm constrained optimization:

$$\begin{aligned} \min_X \quad & \sum_{(i,j) \in \Omega} (X_{i,j} - M_{i,j})^2 \\ \text{s.t.} \quad & \|X\|_* \leq \alpha, \end{aligned} \tag{28}$$

for some hyperparameter $\alpha > 0$.

How well does nuclear norm minimization work? Imagine we want to recover an unknown low-rank matrix $M \in \mathbb{R}^{d_1 \times d_2}$. We observe m entries from M uniformly at random. Then, given another matrix X , we may define the mean squared recovery error as

$$\frac{1}{d_1} \frac{1}{d_2} \sum_{i=1}^{d_1} \sum_{j=1}^{d_2} (X_{i,j} - M_{i,j})^2. \quad (29)$$

Without further assumptions, we would need to see the entire matrix: Imagine the unknown matrix only has an extremely large entry but remains nonzero elsewhere. Without sampling this entry, we are only observing zeros. Therefore, we shall assume that M is bounded in all entries—this is a reasonable assumption for modeling many applications (e.g., movie ratings). In general, one may assume that M is incoherent (meaning that the row norms of the low-rank factors are bounded at the order of $O(n^{-1})$).

Remark 2.18 (Rank-constrained matrix factorization). *An alternative way to formulate the above is by writing it as a convex optimization problem:*

$$\min_{X \in \mathbb{R}^{d_1 \times d_2}} \sum_{(i,j) \in \Omega} (X_{i,j} - M_{i,j})^2 + \lambda \|X\|_*$$

This is a semi-definite program (SDP), which is convex in X . Instead, we may consider the re-parametrized optimization problem:

$$\min_{U \in \mathbb{R}^{d_1 \times r}, V \in \mathbb{R}^{d_2 \times r}} \sum_{(i,j) \in \Omega} (U_i^\top V_j - M_{i,j})^2,$$

where U_i, V_j are both r -dimensional vectors, for any i, j . Put together, $U = [U_1, U_2, \dots, U_{d_1}]$, $V = [V_1, V_2, \dots, V_{d_2}]$ are both rank- r matrices.

2.6.1 Recovery guarantees

Let $\alpha = dr$ and denote by M^* the output of program (28). We will show that the recovery error of M^* given by equation (29) is at most:

$$2r \sqrt{\frac{\log(d)}{md}},$$

where $d = \max(d_1, d_2)$. For example, if we want the test error less than ϵ , then the number of samples needs to be at least

$$m \geq \frac{dr^2}{\epsilon^2 \log(d)}.$$

We will use Rademacher complexity to establish this result. Define the hypothesis class to be the set of nuclear norm bounded and entrywise bounded matrices as follows:

$$\mathcal{H} = \{X \in \mathbb{R}^{d_1 \times d_2} \mid \|X\|_* \leq \alpha, \|X\|_\infty \leq 1\}$$

Then, define the Rademacher complexity of $\mathcal{H}$ as

$$R_n(\mathcal{H}) = \mathbb{E}_{\Omega, \sigma_{1:m}} \left[\sup_{X \in \mathcal{H}} \frac{1}{m} \sum_{(i,j) \in \Omega} \sigma_{i,j} \cdot |X_{i,j} - M_{i,j}| \right].$$

Here we use the following result. Suppose $\mathcal{H}$ is a set of matrices whose nuclear norm is bounded by C :

$$\mathcal{H} = \left\{ X \in \mathbb{R}^{d_1 \times d_2} \mid \|X\|_* \leq C \right\}$$

Then, the Rademacher complexity of nuclear norm bounded matrices is at most

$$R_n(\mathcal{H}) = \mathbb{E} \left[\sup_{X \in \mathcal{H}} \frac{1}{m} \sum_{(i,j) \in \Omega} \sigma_{i,j} |X_{i,j} - M_{i,j}| \right] \leq 2C \sqrt{\frac{\log d}{md}} \quad (30)$$

Recall our result from equation (22), if $m \geq dr^2 \log(d\delta^{-1})/\epsilon^2$, then the above Rademacher complexity is at most ϵ for $\alpha = dr$. Based on equation (22), with probability at least $1 - \delta$, the recovery error of the minimizer must be at most $O(\epsilon)$.

It remains to show that equation (30) is true. By symmetry, the absolute function inside $R_n(\mathcal{H})$ can be removed without changing the value of $R_n(\mathcal{H})$:

$$R_n(\mathcal{H}) = \mathbb{E} \left[\sup_{X \in \mathcal{H}} \frac{1}{m} \sum_{(i,j) \in \Omega} \sigma_{i,j} (X_{i,j} - M_{i,j}) \right]$$

Let us introduce a zero-one matrix Σ such that $\Sigma_{i,j} = \sigma_{i,j}$ if $(i,j) \in \Omega$, and $\Sigma_{i,j} = 0$ otherwise. Hence, the above is equal to

$$\begin{aligned} \mathbb{E}_{\Omega, \sigma_{1:n}} \left[\sup_{X \in \mathcal{H}} \frac{1}{m} \langle \Sigma, X - M \rangle \right] &\leq \mathbb{E} \left[\sup_{X \in \mathcal{H}} \frac{1}{m} \|\Sigma\|_2 \cdot \|X - M\|_* \right] \\ &\leq \frac{2C}{m} \mathbb{E} [\|\Sigma\|_2] \end{aligned} \quad (31)$$

Above we use the fact that both X, M have nuclear norm bounded above by C . Hence, to finish the proof, we are left to deal with the expectation of $\|\Sigma\|_2$. We use the claim that

$$\mathbb{E} [\|\Sigma\|_2] \leq \sqrt{\frac{m \log d}{d}}. \quad (32)$$

Plugging this claim back into equation (31), we have thus shown that

$$R_n(\mathcal{H}) \leq 2C \sqrt{\frac{\log(d)}{md}}.$$

Showing equation (32) requires using some facts related to the spectrum of random matrices. We will consider covering this result at a later point in class.

3 Optimization and generalization in neural networks

In the previous section, we saw the concept of uniform convergence, and showed that with this concept, we can get pretty strong results on a variety of problems, including ℓ_2/ℓ_1 -regularized linear functions, matrix completion, and two-layer neural networks. However, it turns out that moving beyond these settings with the techniques we've developed is quite challenging. A couple of key challenges are:

- In terms of the sample/learning complexity, how could we go beyond two layers? How does this complexity depend on the data distribution?
- How could we incorporate the inductive bias induced by the choice of specific optimization algorithms into the learning complexity?

The goal of this section is to tackle the above questions. In particular, we shall begin by establishing the folklore intuition that the learning complexity of an ML model scales with its number of parameters. This kind of folklore is especially intriguing in the context of deep networks, as these models have millions (and now billions) of parameters. However, this high complexity clearly does not explain/corroborate with the empirical results we typically see with supervised fine-tuning and in-context learning.

3.1 Neural tangent kernels (Lectures 10-12)

The performance of a machine learning model breaks down into two parts:

$$L(\hat{f}) - \inf_f L(f) = \underbrace{L(\hat{f}) - \inf_{f \in F} L(f)}_{\text{Estimation error}} + \underbrace{\inf_{f \in F} L(f) - \inf_f L(f)}_{\text{Approximation error}}$$

Both optimization and generalization results aim to reduce estimation error. Approximation error results, on the other hand, relate to the expressivity of a function class.

Example 3.1 (Linear models and kernel features). *For linear methods (i.e., $f_w(x) = w^\top x$ for some parameter $w \in \mathbb{R}^d$ and input feature vector $x \in \mathbb{R}^d$), the estimation error is small, but the approximation error is large.*

Kernel methods represent one way to move beyond linear methods. In kernel methods, we replace $\langle x, w \rangle$ with $\langle \phi(x), w \rangle$, where $\phi(x)$ is an arbitrary feature map of x . Then, we may use the kernel features in a regression model. Suppose we are minimizing the mean squared error using a kernel method:

$$\ell(w) = \frac{1}{2n} \sum_{i=1}^n (\langle \phi(x_i), w \rangle - y_i)^2$$

Definition 3.2 (Kernels). *A function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a positive semi-definite kernel if and only if for every subset of inputs $\{x_1, x_2, \dots, x_n\} \subseteq \mathcal{X}$, the matrix $K \in \mathbb{R}^{n \times n}$ defined by using the kernel map*

$$K_{i,j} = k(x_i, x_j), \text{ for every } 1 \leq i, j \leq n$$

*is a positive semi-definite matrix.*⁸

⁸Recall that a symmetric matrix K is positive semi-definite if for every x , $x^\top K x \geq 0$.

Example 3.3. *Linear kernels:* $k(x, x') = \langle x, x' \rangle$. This follows from the fact that $K = XX^\top$ is positive semi-definite.

Gaussian kernels: $k(x, x') = \exp\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)$. This is followed by the fact that either the sum or the product of two kernels (the element-wise product (Hadamard product) of two PSD matrices is still PSD) is still a kernel.

Polynomial kernels: $k(x, x') = (1 + \langle x, x' \rangle)^p$.⁹

An alternative definition of kernels is through feature maps. Recall that a feature map ϕ maps an input x to a feature embedding $\phi(x)$. Consider the kernel $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ defined by

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

This must be a kernel. To show this, we will follow the definition of a kernel by taking a vector α and let

$$\alpha^\top K \alpha = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \langle \phi(x_i), \phi(x_j) \rangle = A^\top A,$$

where $A = \sum_{i=1}^n \alpha_i \phi(x_i)$.

3.1.1 Motivation

It has been widely observed that gradient-based optimization algorithms often converge to small training errors on a complex neural network model. A widely believed explanation for this surprising phenomenon is that the neural net is over-parametrized. The neural tangent kernel represents one of the earliest attempts to mathematically formulate the theory of over-parameterized neural networks. The neural tangent kernel arises from the dynamics of the predictions applied to the training data.

First, we state the problem setup. Given a training dataset $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, let $w \in \mathbb{R}^N$ represent the parameters of a neural network. Let $f_w(x)$ denote the output of the network. We shall restrict our attention to the mean squared loss (for solving regression problems)

$$\ell(w) = \frac{1}{2} \sum_{i=1}^n (f_w(x_i) - y_i)^2$$

For the purpose of illustration, let us consider the gradient flow update. Let $w(t)$ denote the iterate configuration at time $t \geq 0$. Then, $w(t)$ evolves according to

$$\frac{dw(t)}{dt} = -\nabla \ell(w(t))$$

We now show that the learning dynamics are characterized by a positive semi-definite matrix $H(t) \in \mathbb{R}^{n \times n}$, defined as

$$H_{i,j}(t) = \left\langle \frac{\partial f_{w(t)}(x_i)}{\partial w_i}, \frac{\partial f_{w(t)}(x_j)}{\partial w_j} \right\rangle \quad (33)$$

Then, let the network outputs at time t be defined as

$$u(t) = [f_{w(t)}(x_1), f_{w(t)}(x_2), \dots, f_{w(t)}(x_n)],$$

⁹Exercise: verify that both the polynomial kernel and the Gaussian kernel are indeed kernels

for any $t \geq 0$. Finally, let $y = [y_1, y_2, \dots, y_n]$. Then, we will show that the network outputs will follow the dynamics as

$$\frac{du(t)}{dt} = -H(t)(u(t) - y)$$

Proof. Recall that the evolution of the network parameters follows an update as

$$\frac{dw(t)}{dt} = -\nabla \ell(w(t)) = -\sum_{i=1}^n (f_{w(t)}(x_i) - y_i) \frac{df_{w(t)}(x_i)}{dw}$$

Using chain rule and multi-variate calculus, the evolution of the network output satisfies

$$\frac{df_{w(t)}(x_i)}{dt} = \left\langle \frac{df_{w(t)}(x_i)}{dw}, \frac{dw(t)}{dt} \right\rangle$$

By applying the first step to the second step, we get

$$\begin{aligned} \frac{df_{w(t)}(x_i)}{dt} &= \left\langle \frac{df_{w(t)}(x_i)}{dw}, -\sum_{j=1}^n (f_{w(t)}(x_j) - y_j) \frac{df_{w(t)}(x_j)}{dw} \right\rangle \\ &= -\sum_{j=1}^n (f_{w(t)}(x_j) - y_j) \underbrace{\left\langle \frac{df_{w(t)}(x_i)}{dw}, \frac{df_{w(t)}(x_j)}{dw} \right\rangle}_{i, j\text{-th entry of the } H(t) \text{ matrix}} \\ &= -H(t)(u(t) - y) \end{aligned}$$

In summary, the learning dynamics are captured by a symmetric matrix $H(t)$. Now, how does the NTK arise? We will define an ultra-wide neural net whose width goes to infinity. In the limit, it can be shown that the matrix $H(0)$ remains constant during training, i.e., equal to $H(0)$. Moreover, under a random initialization of parameters, $H(0)$ converges to a deterministic kernel matrix H^* , the neural tangent kernel (NTK).

3.1.2 Definition

Consider the mapping of a two-layer neural network

$$x \rightarrow \sum_{i=1}^m a_i \sigma(w_i^\top x), \quad (34)$$

where σ represents a nonlinear activation function (such as ReLU or sigmoid). Now consider the mapping rescaled by $\frac{1}{\sqrt{m}}$:

$$x \rightarrow \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x)$$

We shall linearize the right-hand side around the initialization $w_i(0)$, for all $i = 1, 2, \dots, m$; Essentially, performing Taylor's expansion to derive the following

$$\begin{aligned} x &\rightarrow \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \left(\sigma((w_i(0))^\top x) + (w_i - w_i(0))^\top x \sigma'((w_i(0))^\top x) \right) \\ &= \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \left(\sigma((w_i(0))^\top x) - (w_i(0))^\top x \sigma'((w_i(0))^\top x) \right) + \frac{1}{\sqrt{m}} x^\top \left(\sum_{i=1}^m a_i w_i \sigma'((w_i(0))^\top x) \right) \end{aligned}$$

Next, recall that the dynamics of predictions is governed by $H(t) \in \mathbb{R}^{n \times n}$ (see equation (33)). Recall that f is given in equation (34). Therefore, the i, j -th entry of the kernel matrix $H(t)$ is

$$H_{i,j}(0) = x_i^\top x_j \sum_{r=1}^m \left\langle \frac{a_r \sigma'((w_r(0)^\top x_i))}{\sqrt{m}}, \frac{a_r \sigma'((w_r(0)^\top x_j))}{\sqrt{m}} \right\rangle$$

Suppose we sample every $w_r(0)$ from a standard Gaussian distribution. In addition, suppose we sample a_r uniformly between $\{+1, -1\}$. Then, one can view the above as the average of m independent random variables. When m becomes very large, then by the law of large numbers, the average is close to its expectation. This gives rise to the neural tangent kernel evaluated at x_i, x_j as follows:

$$H_{i,j}^* = x_i^\top x_j \mathbb{E}_{w \sim \mathcal{N}(0, \text{Id})} \left[\sigma'(w^\top x_i) \sigma'(w^\top x_j) \right]. \quad (35)$$

Example 3.4 (Two-layer ReLU networks with second-layer fixed). *Consider a two-layer network $f(x) = \frac{1}{\sqrt{m}} \sum_{r=1}^m a_r \sigma(w_r^\top x)$, where $\sigma(z) = \max(0, z)$, $a_r \in \{\pm 1\}$, and $w_r(0) \sim \mathcal{N}(0, \text{Id}_d)$. For ReLU, the derivative is $\sigma'(z) = \mathbb{1}_{z>0}$. Hence, the kernel map between two inputs x, x' is*

$$H^*(x, x') = x^\top x' \mathbb{E}_{w \sim \mathcal{N}(0, \text{Id}_d)} [\mathbb{1}_{w^\top x > 0} \cdot \mathbb{1}_{w^\top x' > 0}].$$

Let θ denote the angle between x and x' such that $\cos \theta = \frac{x^\top x'}{\|x\| \cdot \|x'\|}$. For a Gaussian random vector, the pair $(w^\top x, w^\top x')$ is jointly normal, and

$$\Pr(w^\top x > 0, w^\top x' > 0) = \frac{1}{4} + \frac{1}{2\pi} \arcsin(\cos \theta) = \frac{\pi - \theta}{2\pi},$$

using the fact that $\arcsin(\cos \theta) = \frac{\pi}{2} - \theta$ for $\theta \in [0, \pi]$. Another argument is that since w is rotation-invariant, we can essentially project x, x' to a two-dimensional plane, and instead imagine that we are drawing w from a unit circle. Therefore, the NTK becomes $H^*(x, x') = x^\top x' \cdot \frac{\pi - \theta}{2\pi}$.

As an example, let $x_1 = (1, 0)$ and $x_2 = (0, 1)$ in $\mathbb{R}^2$. Then $x_1^\top x_2 = 0$, and $H^*(x_1, x_2) = 0$. Since x_1 and x_2 are both unit vectors, we also have $H^*(x_1, x_1) = H^*(x_2, x_2) = \frac{1}{2}$. Thus, the NTK matrix for x_1, x_2 is $H^* = [\frac{1}{2}, 0; 0, \frac{1}{2}]$.

Example 3.5 (Kernel ridge regression). *One way to apply the above kernel feature map is via kernel ridge regression (KRR), which learns a function in the associated reproducing kernel Hilbert space (RKHS) by solving*

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n (f(x_i) - y_i)^2 + \lambda \|f\|_{\mathcal{H}}^2, \quad (36)$$

where $\lambda > 0$ is a regularization parameter. Based on the representer theorem, the optimal solution admits the finite-dimensional form

$$f(x) = \sum_{i=1}^n \alpha_i k(x_i, x),$$

for some coefficients $\alpha \in \mathbb{R}^n$. Let $K \in \mathbb{R}^{n \times n}$ be the kernel matrix with $K_{i,j} = k(x_i, x_j)$, and let $y = (y_1, \dots, y_n)^\top$. Substituting the representation into objective (36) above gives the quadratic problem—specifically, setting $\|f\|_{\mathcal{H}}^2 = \alpha^\top K \alpha$ gives

$$\min_{\alpha \in \mathbb{R}^n} \|K \alpha - y\|^2 + \lambda \alpha^\top K \alpha,$$

whose solution is $\alpha = (K + \lambda \text{Id})^{-1}y$ (assuming that K is full-rank).

For a new input x , the prediction is $f(x) = k_x^\top \alpha$, where $k_x = (k(x_1, x), \dots, k(x_n, x))^\top$. In the NTK regime, the kernel $k(x, x')$ is given by the neural tangent kernel $H^*(x, x')$. Training a sufficiently wide neural network with squared loss is equivalent to performing kernel ridge regression with kernel matrix H^* .

Specifically, the feature map of any input x_i under the two-layer ReLU NTK is given by $[\frac{a_1 \sigma'(w_1(0)^\top x_i)}{\sqrt{m}} x_i, \dots, \frac{a_m \sigma'(w_m(0)^\top x_i)}{\sqrt{m}} x_i]$, for every $i = 1, 2, \dots, n$. The H^* kernel is given in equation (35).

3.1.3 Convergence analysis (optional)

Based on the NTK defined above, we now show that for this particular initialization, the neural network stays close to initialization long enough so that the loss decreases to a sufficiently low level. There are essentially two steps here:

- Step 1: For a sufficiently wide network, the randomly initialized neural net is close to the expectation – the NTK.
- Step 2: For a sufficiently wide network, the kernel matrix at time t remains close to the initialization.

For reference, much of the proof in this section is taken from the monograph by Arora et al., 2020.

$H(0)$ is close to H^* for sufficiently large m : Suppose the activation function (e.g., ReLU) is 1-Lipschitz continuous. Suppose for every input x_i , the Euclidean norm of x_i is less than 1, for any $i = 1, 2, \dots, n$. For any $\epsilon > 0$, if $m \geq O(n^2 \log(\delta^{-1})/\epsilon^2)$ (recall n is the number of samples at the input), then with probability at least $1 - \delta$, we have that

$$\|H(0) - H^*\|_F \leq \epsilon$$

To show that this is true, we will first show that every entry of $H(0)$ is close to H^* with probability $1 - O(\delta/n^2)$. First, we can see that every individual entry of $H(0)$ is bounded from above by 1:

$$\left| x_i^\top x_j \cdot \sigma'((w_r(0))^\top x_i) \sigma'((w_r(0))^\top x_j) \right| \leq 1$$

Now we shall apply Hoeffding's inequality (see (13)), and set the deviation as ϵ/n . Then, the failure probability becomes

$$2 \exp\left(-\frac{2\epsilon^2 m}{n^2}\right)$$

When $m \geq O(\frac{n^2 \log(n\delta^{-1})}{\epsilon^2})$, the above probability can be reduced below $O(\delta/n^2)$. By taking a union bound over all the entries of $H(0)$, we have that the above will hold with probability $1 - \delta$.

Finally, we convert the entry-wise error bound to the operator norm bound:

$$\|H(0) - H^*\|_F \leq \sqrt{n^2 \cdot \frac{\epsilon^2}{n^2}} = \epsilon$$

$H(t)$ remains close to H^* : Suppose that $y_i = O(1)$ for all $i = 1, 2, \dots, n$. Let $t > 0$, suppose that for all $0 < s < t$, $u_i(s) = O(1)$ for all $i = 1, 2, \dots, n$. Then if $m \geq O(\frac{n^6 t^2 \log(n\delta^{-1})}{\epsilon^2})$, with probability $1 - \delta$,

$$\|H(t) - H^*\|_F \leq \epsilon$$

There are two steps in the proof of this result:

- We first show that every weight vector remains close to the initialization if the width is large.
- We then show that this implies the kernel matrix remains close to the NTK.

Consider the movement of a single weight vector w_r :

$$\begin{aligned} \|w_r(t) - w_r(0)\| &= \left\| \int_0^t \frac{dw_r(\tau)}{d\tau} d\tau \right\| \\ &= \left\| \int_0^t \frac{1}{\sqrt{m}} \sum_{i=0}^n (u_i(\tau) - y_i) a_r x_i \sigma'(w_r(\tau)^\top x_i) d\tau \right\| \\ &\leq \int_0^t \left\| \frac{1}{\sqrt{m}} \sum_{i=0}^n (u_i(\tau) - y_i) a_r x_i \sigma'(w_r(\tau)^\top x_i) d\tau \right\| \\ &\leq \frac{1}{\sqrt{m}} \sum_{i=0}^n \int_0^t \|(u_i(\tau) - y_i) a_r x_i \sigma'(w_r(\tau)^\top x_i)\| d\tau \leq O\left(\frac{tn}{\sqrt{m}}\right), \end{aligned}$$

where we use the fact that the inputs x_i, y_i and $a_r, u_i(\tau)$ are all bounded by some constants, and the derivative of σ is also bounded by a constant (e.g., sigmoid).

Consider the movement of a single entry of the kernel matrix

$$\begin{aligned} H_{i,j}(t) - H_{i,j}(0) &= \frac{1}{m} \left| \sum_{r=1}^m \left(\sigma'(w_r(t)^\top x_i) \sigma'(w_r(t)^\top x_j) - \sigma'(w_r(0)^\top x_i) \sigma'(w_r(0)^\top x_j) \right) \right| \\ &\leq \frac{1}{m} \left| \sum_{r=1}^m \sigma'(w_r(t)^\top x_i) (\sigma'(w_r(t)^\top x_j) - \sigma'(w_r(0)^\top x_j)) \right| \\ &\quad + \frac{1}{m} \left| \sum_{r=1}^m \sigma'(w_r(0)^\top x_j) (\sigma'(w_r(t)^\top x_i) - \sigma'(w_r(0)^\top x_i)) \right| \\ &\lesssim \frac{1}{m} \sum_{r=1}^m \max_{i=1}^n \|\sigma'(w_r(t)^\top x_i) - \sigma'(w_r(0)^\top x_i)\|_2 \\ &\leq O\left(\frac{tn}{\sqrt{m}}\right) \end{aligned}$$

Taken together, we have shown that if m is large enough, then $H(t)$ will stay close to H^* throughout the entire gradient descent dynamic.

In summary, the dynamics of network outputs are governed by the following approximation using the NTK matrix H^* :

$$\frac{du(t)}{dt} \approx -H^*(u(t) - y)$$



Figure 3: Illustration of the loss and the projected values onto the eigen-space.

We use this to describe two implications: i) Convergence to global minima; ii) Explaining why NTK converges faster using original labels than using random labels.

Notice that this is a linear dynamical system. Denote the eigen-decomposition of H^* as $\sum_{i=1}^n \lambda_i v_i v_i^\top$. Now we consider the dynamics of $u(t)$ on each eigenvector separately. Consider one eigenvector v_i for instance. We have

$$\frac{dv_i^\top u(t)}{dt} \approx -v_i^\top H^*(u(t) - y) = -\lambda_i v_i^\top u(t) + \lambda_i v_i^\top y,$$

which is equivalent to (since y does not depend on $u(t)$)

$$\frac{dv_i^\top (u(t) - y)}{dt} = -\lambda_i v_i^\top (u(t) - y)$$

One function that satisfies this differential equation is when

$$v_i^\top (u(t) - y) = \exp(-\lambda_i t), \text{ for every } i = 1, 2, \dots, n.$$

Notice that $u(t) - y$ is the difference between predictions and training labels at time t .

We would like to show that the difference converges to zero when t is large. The above implies that each component of $u(t) - y$ projected to eigenvector v_i converges to zero exponentially fast at a rate of $\exp(-\lambda_i t)$.

Also notice that the eigenvectors $\{v_i\}_i$ form an orthonormal basis. Since we know that each component converges to zero, together they imply that $u(t) - y$ must also converge to zero. **This shows that the outputs will eventually converge to the training labels, meaning the training loss will go to zero.**

Next, we can see that each component goes to zero at different rates. For larger eigenvalues, the convergence is faster. Hence in order to have fast convergence, we would like the projections of the labels onto the top eigenvectors to be large.

For a set of labels, if they align with top eigenvectors, then gradient descent converges quickly. On the other hand, if the projections are uniform, or aligns with eigenvectors with small eigenvalues, then gradient descent converges with a slow rate. In Figure 3, taken from Neyshabur et al., 2017, we compare convergence rates of gradient descent between using original labels, random labels, and worst-case labels. This experiment is performed to separate two classes of digits from MNIST, and it uses the forward model as $f(a, w) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma'(w_i^\top x)$.

3.2 Implicit regularization (Lectures 13)

Recall that we discussed the matrix completion problem in Section 2.6. Now, we shall study the gradient descent dynamic of this problem in an over-parameterized setting, which generally refers to scenarios where the number of model parameters is far larger than the number of samples.

Example 3.6 (Linear regression). *Suppose we are conducting linear regression on n input samples:*

$$\ell(w) = \frac{1}{n} \sum_{i=1}^n (x_i^\top w - y_i)^2$$

Consider gradient descent:

$$w_{t+1} = w_t - \eta \nabla \ell(w_t)$$

Suppose n is smaller than the dimension of $w \in \mathbb{R}^d$. Suppose $w_0 = 0$. Then, given that $\nabla \ell(w_t) = \frac{2}{n} \sum_{i=1}^n (x_i^\top w_t - y_i) x_i$, w_{t+1} will always lie inside the span of $\{x_1, x_2, \dots, x_n\}$, whose dimension is smaller than d . In other words, gradient descent searches inside a space whose dimension is much smaller than the actual dimension of the entire space.

To analyze convergence, let $u_t = Xw_t$ be the predictions at time t . Multiplying the update rule by X , we get:

$$u_{t+1} = u_t - \frac{2\eta}{n} XX^\top (u_t - y)$$

Subtracting y from both sides, the residual error $r_t = u_t - y$ evolves as:

$$r_{t+1} = \left(\text{Id} - \frac{2\eta}{n} XX^\top\right) r_t$$

where $XX^\top \in \mathbb{R}^{n \times n}$. Thus, if the step size satisfies $\eta < 2/\lambda_{\min}(XX^\top/n)$, then the residual error at step t satisfies:

$$\|u_t - y\| \leq \left(1 - 2\eta \lambda_{\min}\left(\frac{XX^\top}{n}\right)\right)^t \|u_0 - y\|$$

In the over-parameterized case ($d > n$), if the rank of X is full (equal to n), then $\lambda_{\min}(XX^\top/n)$ is strictly bounded away from zero, and the training error converges to zero at a geometric rate.

Example 3.7 (Gradient descent for minimizing smooth functions). *The above is part of a broader phenomenon concerning the minimization of a smooth, possibly nonconvex, unconstrained function.*

A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is smooth if the gradient of f satisfies $\|\nabla f(x) - \nabla f(y)\| \leq C \cdot \|x - y\|$ for a fixed constant $C > 0$, given any x, y within some continuous domain.

One can show that gradient descent converges to a stationary point x such that $\|\nabla f(x)\| \leq \epsilon$ within $O(\epsilon^{-2})$ iterations (Ghadimi and Lan, 2013), where the big- O notation folds the diameter of the search space and other problem-dependent constants including the smoothness parameter C .

Next, we generalize the above to a matrix factorization setup. Suppose there is an underlying rank- r matrix $X^* \in \mathbb{R}^{d \times d}$. In particular, for the illustration of the analysis, we shall focus on the case of X^* being positive semi-definite. As for the input to the

problem, suppose we have m random Gaussian matrices whose entries are all drawn from $\mathcal{N}(0, 1)$, denoted as $A_1, A_2, \dots, A_m$, all with dimension d by d . In addition, suppose they are symmetric, for example, we could transform each A_i as $(A_i + A_i^\top)/2$. Let $y_i = \langle A_i, UU^\top \rangle$, for $i = 1, 2, \dots, n$.

A solution would be to find another positive semi-definite matrix $UU^\top$ so that the distance between $UU^\top$ and X^* is minimized. In particular, we shall consider the case that $U \in \mathbb{R}^{d \times d}$. Then, we set up a mean squared loss as follows:

$$\ell(U) = \frac{1}{n} \sum_{i=1}^n \left(\langle UU^\top, A_i \rangle - y_i \right)^2$$

In practice, we see that this loss function has many solutions that render its loss to near zero, even though $UU^\top$ can be quite far from X^* . A key observation is that the generalization behavior of U should depend on the initialization. Next we shall make this precise by analyzing the gradient descent dynamic. For reference, this proof is based on the work of Li et al., 2018.

Let $U_0 = \alpha \text{Id}$, for some small values of α . Then, let the gradient descent update be given by

$$U_{t+1} = U_t - \eta \nabla \ell(U_t) = (\text{Id} - \eta M_t) U_t,$$

where

$$M_t = \frac{1}{m} \sum_{i=1}^m \langle A_i, U_t U_t^\top - X^* \rangle A_i$$

We shall restrict the analysis to the rank-1 case where $r = 1$, where we can write $X^* = u^* u^{*\top}$, for some $u^* \in \mathbb{R}^d$. Let $\text{Id}_{u^*} = \text{Id} - \frac{u^* u^{*\top}}{\|u^*\|^2}$. For simplicity, let's take the norm of u^* as 1. Then, we decompose the iterates U_t into two parts into the subspace of u^* and its complement:

$$\begin{aligned} U_t &= u^* u^{*\top} U_t + (\text{Id} - \text{Id}_{u^*}) U_t \\ &:= u^* r_t^\top + E_t, \end{aligned}$$

where we denote by $r_t = U_t^\top u^*$, and $E_t = (\text{Id} - \text{Id}_{u^*}) U_t$.

We will show that the spectral norm and the Frobenius norm of the “error term” E_t remains small throughout the iterations, whereas the “signal” term r_t grows exponentially fast (in the sense that the norm of r_t grows to the norm of u^* .) Note that any solution with $\|r_t\| = 1$ and $E_t = 0$ will give exact recovery, and for the purpose of this section we will show that $\|r_t\|$ will converge approximately to 1 and E_t stays small.

Based on the gradient update above, we derive the update for E_t :

$$\begin{aligned} E_{t+1} &= (\text{Id} - \text{Id}_{u^*})(\text{Id} - \eta M_t) U_t \\ &= E_t - \eta (\text{Id} - \text{Id}_{u^*}) M_t U_t \end{aligned} \tag{37}$$

We shall also assume that the set of measurement matrices $(A_1, A_2, \dots, A_m)$ satisfies the restricted isometry property. A set of linear measurement matrices $A_1, \dots, A_m$ satisfies (r, δ) -restricted isometry property (RIP) if for any d by d matrix X with rank at most r , we have that

$$(1 - \delta) \|X\|_F^2 \leq \frac{1}{m} \sum_{i=1}^m \langle A_i, X \rangle^2 \leq (1 + \delta) \|X\|_F^2 \tag{38}$$

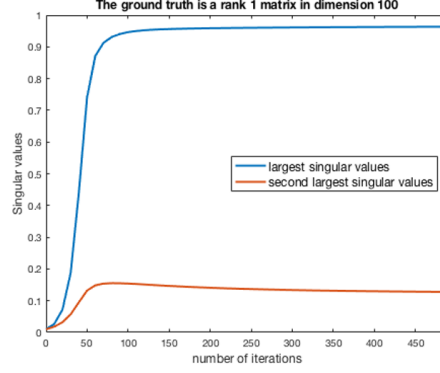


Figure 4: Illustrating the spectral separation between E_t and U_t during the gradient descent dynamic in the case of over-parametrized rank-1 matrix sensing.

The central thesis is to show a spectral separation between E_t and U_t . For an illustration, see Figure 4, taken from Li et al., 2018. In the following, we shall assume that the set of measurement matrices satisfies $(4, \delta)$ -RIP with $\delta \leq c$ (for some small enough constant c). One corollary of the above definition is that for any matrices $X, Y \in \mathbb{R}^{d \times d}$ with rank at most r , we also have:

$$\left| \frac{1}{m} \sum_{i=1}^m \langle A_i, X \rangle \langle A_i, Y \rangle - \langle X, Y \rangle \right| \leq \delta \|X\|_F \|Y\|_F \quad (39)$$

If Y is at most rank r but X may not be, then we have the following instead:

$$\left| \frac{1}{m} \sum_{i=1}^m \langle A_i, X \rangle \langle A_i, Y \rangle - \langle X, Y \rangle \right| \leq \delta \|X\|_* \|Y\|_F \quad (40)$$

Theorem 3.8 (Convergence of gradient descent from small initialization in over-parametrized matrix sensing). *Suppose $\alpha \leq \delta \sqrt{\frac{1}{d} \log(\frac{1}{\delta})}$ and $\eta \lesssim c\delta^2 \log^{-1}(\frac{1}{\delta\alpha})$. Then after $T = \Theta(\frac{(\alpha\delta)^{-1}}{\eta})$ iterations, we have that*

$$\left\| U_T U_T^\top - X^* \right\|_F \lesssim \delta \log(\delta^{-1}) \quad (41)$$

3.2.1 Error dynamics (optional)

Based on the update rule (37) for E_t , we have that

$$\|E_{t+1}\|_F^2 = \|E_t\|_F^2 - 2\eta \langle E_t, (\text{Id} - \text{Id}_{u^*}) M_t U_t \rangle + \eta^2 \|(\text{Id} - \text{Id}_{u^*}) M_t U_t\|_F^2$$

When η is sufficiently small, and $\|M_t\|_F, \|U_t\|_2$ are both bounded from above, the third term on the right-hand side is negligible compared to the second term. Therefore, we focus on the second term.

$$\begin{aligned} \langle E_t, (\text{Id} - \text{Id}_{u^*}) M_t U_t \rangle &= \frac{1}{m} \sum_{i=1}^m \langle A_i, U_t U_t^\top - X^* \rangle \langle A_i, (\text{Id} - \text{Id}_{u^*}) E_t U_t^\top \rangle \\ &= \frac{1}{m} \sum_{i=1}^m \langle A_i, U_t U_t^\top - X^* \rangle \langle A_i, E_t U_t^\top \rangle \end{aligned}$$

We use (39) to analyze the above, but one gap here is that $U_t U_t^\top$ is not necessarily low-rank. Thus, we decompose it into a low-rank part and an error part with small trace norm.

$$\frac{1}{m} \sum_{i=1}^m \langle A_i, U_t U_t^\top - X^\star - E_t E_t^\top \rangle \langle A_i, E_t r_t u^{\star\top} \rangle \quad (42)$$

$$\geq \langle U_t U_t^\top - X^\star - E_t E_t^\top, E_t r_t u^{\star\top} \rangle - \delta \left\| U_t U_t^\top - X^\star - E_t E_t^\top \right\|_F \left\| E_t r_t u^{\star\top} \right\|_F \quad (43)$$

$$\geq \langle U_t U_t^\top - X^\star - E_t E_t^\top, E_t r_t u^{\star\top} \rangle - 2\delta \|E_t r_t\| \quad (44)$$

Above we use the fact that $\|U_t U_t^\top - X^\star - E_t E_t^\top\|_F, \|r_t\|, \|u^\star\|$ are all bounded from above by 1. As for the $E_t E_t^\top$ part, we first note that $\langle A_i, E_t E_t^\top \rangle^2 \geq 0$. Thus, we can focus on the cross terms.

$$\begin{aligned} \frac{1}{m} \sum_{i=1}^m \langle A_i, E_t E_t^\top \rangle \langle A_i, E_t r_t u^{\star\top} \rangle &\geq \langle E_t E_t^\top, E_t r_t u^{\star\top} \rangle - \delta \left\| E_t E_t^\top \right\|_\star \left\| E_t r_t u^{\star\top} \right\| \\ &\geq \langle E_t E_t^\top, E_t r_t u^{\star\top} \rangle - \frac{\delta}{2} \|E_t r_t\|, \end{aligned}$$

where we are using the fact that $\|E_t E_t^\top\|_\star$ is bounded from above by some constant. The other cross term would be

$$\begin{aligned} \frac{1}{m} \sum_{i=1}^m \langle A_i, E_t E_t^\top \rangle \langle A_i, U_t U_t^\top - X^\star - E_t E_t^\top \rangle \\ \geq \langle E_t E_t^\top, U_t U_t^\top - X^\star - E_t E_t^\top \rangle - \delta \left\| E_t E_t^\top \right\|_\star \left\| U_t U_t^\top - X^\star - E_t E_t^\top \right\|_F \\ \geq \langle E_t E_t^\top, U_t U_t^\top - X^\star \rangle - \delta \left\| E_t E_t^\top \right\|_\star \end{aligned}$$

Because $E_t^\top X^\star = 0$ and also $\|U_t U_t^\top - X^\star - E_t E_t^\top\|_F$ is bounded from above by 1. All put together, we can conclude that

$$\langle E_t, (\text{Id} - \text{Id}_{u^\star}) M_t U_t \rangle \geq \langle U_t U_t^\top - X^\star, E_t U_t^\top \rangle - 2.5\delta \|E_t r_t\| - \delta \|E_t\|_F^2$$

Next, we have

$$\begin{aligned} \langle U_t U_t^\top - X^\star, E_t U_t^\top \rangle &= \langle U_t U_t^\top, E_t U_t^\top \rangle = \langle U_t^\top, U_t^\top E_t U_t^\top \rangle \\ &= \langle U_t^\top, E_t^\top E_t U_t^\top \rangle \\ &= \langle E_t U_t^\top, E_t U_t^\top \rangle \geq 0 \end{aligned}$$

In summary, we have proved that

$$\|E_{t+1}\|_F^2 \leq (1 + \eta\delta) \|E_t\|_F^2 + 2.5\delta\eta \|E_t r_t\| + O(\eta^2) \quad (45)$$

We could also control the growth of the largest singular value of E_t as

$$E_{t+1} = E_t - \eta(\text{Id} - \text{Id}_{u^\star}) M_t U_t \quad (46)$$

$$= E_t - \eta(\text{Id} - \text{Id}_{u^\star}) E_t U_t^\top U_t + \eta(\text{Id} - \text{Id}_{u^\star})(M_t U_t - E_t U_t^\top U_t) \quad (47)$$

Next, notice that $(\text{Id} - \text{Id}_{u^*})(U_t U_t^\top - X^*)U_t = E_t U_t^\top U_t$, and

$$\begin{aligned} & \left\| (\text{Id} - \text{Id}_{u^*})M_t U_t - (\text{Id} - \text{Id}_{u^*})(U_t U_t^\top - X^*)U_t \right\|_2 \\ & \leq 4\delta \left(\left\| U_t U_t^\top - X^* - E_t E_t^\top \right\|_F + \left\| E_t E_t^\top \right\|_* \right) \|U_t\|_2 \\ & \leq 8\delta \|U_t\|_2 \leq 8\delta(\|r_t\| + \|E_t\|_2) \end{aligned}$$

Here, we are using two more properties about RIP measurements: if X is at most rank r , then

$$\left\| \frac{1}{m} \sum_{i=1}^m \langle A_i, X \rangle A_i R - X R \right\|_2 \leq \delta \|X\|_F \|R\|_2 \quad (48)$$

In addition, for any X , we have

$$\left\| \frac{1}{m} \sum_{i=1}^m \langle A_i, X \rangle A_i R - X R \right\|_2 \leq \delta \|X\|_* \|R\|_2 \quad (49)$$

The same is true if we have another matrix U where we insert before A_i :

$$\left\| \frac{1}{m} \sum_{i=1}^m \langle A_i, X \rangle U A_i R - U X R \right\|_2 \leq \delta \|U\|_2 \|X\|_* \|R\|_2 \quad (50)$$

Therefore, we can conclude that

$$\begin{aligned} \|E_{t+1}\|_2 & \leq \left\| E_t (\text{Id} - \eta U_t^\top U_t) \right\|_2 + 2\eta\delta(\|r_t\| + \|E_t\|_2) \\ & \leq (1 + 2\eta\delta) \|E_t\|_2 + 2\eta\delta \|r_t\| \end{aligned}$$

3.2.2 Signal dynamics (optional)

Next, we show that

$$\left\| r_{t+1} - (1 + \eta(1 - \|r_t\|^2))r_t \right\| \leq \eta \|E_t\|_2^2 \|r_t\| + 2\eta\delta(\|E_t\|_2 + \|r_t\|) \quad (51)$$

By the update rule $U_{t+1} + (\text{Id} - \eta M_t)U_t$. We have that

$$r_{t+1} = U_{t+1}^\top u^* = U_t^\top (\text{Id} - \eta M_t^\top) u^* = r_t - \eta U_t^\top M_t^\top u^*$$

Recall that $M_t = \frac{1}{m} \sum_{i=1}^m \langle A_i, U_t U_t^\top - X^* \rangle A_i$. Using (48) and (50), we have that

$$\left\| r_{t+1} - (r_t - \eta U_t^\top (U_t U_t^\top - X^*) u^*) \right\| \leq \delta \left(\left\| U_t U_t^\top - X^* - E_t E_t^\top \right\|_F + \left\| E_t E_t^\top \right\|_* \right) \|U_t\|_2 \quad (52)$$

Here we observe that

$$U_t^\top (U_t U_t^\top - X^*) u^* = U_t^\top U_t r_t - r_t = (r_t r_t^\top + E_t^\top E_t) r_t - r_t = (\|r_t\|^2 - 1) r_t - E_t^\top E_t r_t \quad (53)$$

Therefore, we have that

$$\left\| r_{t+1} - (1 + \eta(1 - \|r_t\|^2))r_t \right\| \leq \eta \left\| E_t^\top E_t r_t \right\| + 2\eta\delta \|U_t\|_2 \quad (54)$$

The above suggests that r_t will eventually drift towards a vector whose norm is equal to one.

Putting things together, we essentially showed that E_t will remain relatively smaller than r_t , since E_t only grows by a rate of $1 + \eta\delta$. Meanwhile, r_t grows by a rate of $1 + O(\eta)$, when it is still bounded away from 1.

3.3 Benign overfitting (Lecture 13)

Benign overfitting represents yet another approach to explain the strong generalization of deep neural networks (Bartlett et al., 2020). The basic hypothesis behind **benign overfitting** stipulates that for a prediction rule $\hat{f}$, we can decompose it into

$$\hat{f} = \hat{f}_0 + \Delta,$$

where $\hat{f}_0$ is a simple component useful for prediction, Δ is a useful component useful for benign overfitting. While Δ is not useful for prediction, it is benign. In the setting of over-parameterized matrix sensing from the previous section, we see that $u^\star r_t^\top$ corresponds to $\hat{f}_0$, whereas E_t relates to Δ .

To better understand the phenomenon of benign overfitting, we shall now consider the case of linear regression.

- We denote the covariates as $x \in \mathbb{R}^d$ and the label as $y \in \mathbb{R}$.
- We assume that x is sub-Gaussian, meaning that there exists $c > 0$ such that $\Pr \left[\|x\|^2 > c \mathbb{E} \left[\|x\|^2 \right] \right] \leq \delta$. Moreover, $\mathbb{E}[y | x] = x^\top \theta^\star$.
- Let us define

$$\begin{aligned} \Sigma &= \mathbb{E} \left[x x^\top \right] = \sum_{i=1}^d \lambda_i v_i v_i^\top, \text{ for } i = 1, 2, \dots, d \\ \theta^\star &= \arg \min_{\theta} \mathbb{E} \left[(y - x^\top \theta)^2 \right] \\ \sigma^2 &= \mathbb{E} \left[(y - x^\top \theta^\star)^2 \right] \end{aligned}$$

A key concept here to understand benign overfitting is the *minimum norm interpolator*. Let $X \in \mathbb{R}^{n \times d}$ denote the features of n samples. Let $y = [y_1, y_2, \dots, y_n]$ denote the labels. We have the estimator $\hat{\theta} = (X^\top X)^\dagger X^\top y$, which solves the problem

$$\begin{aligned} \min_{\theta} \|\theta\|^2 \\ \text{s.t. } \|X\theta - y\|^2 = \min_{\theta'} \|X\theta' - y\|^2 \end{aligned}$$

Now we define the excess prediction risk of $\hat{\theta}$ as

$$\begin{aligned} R(\hat{\theta}) &:= \mathbb{E}_{x,y} \left[(y - x^\top \hat{\theta})^2 \right] - \min_{\theta} \mathbb{E}_{x,y} \left[(y - x^\top \theta)^2 \right] \\ &= \mathbb{E}_{x,y} \left[(y - x^\top \hat{\theta})^2 - (y - x^\top \theta^\star)^2 \right] \\ &= (\hat{\theta} - \theta^\star)^\top \Sigma (\hat{\theta} - \theta^\star), \end{aligned}$$

where we are using the fact that $\mathbb{E}[x x^\top] = \Sigma$ and $\mathbb{E}[y|x] = x^\top \theta^\star$. Therefore, Σ determines the importance of parameter directions. Here is the question: when can the label noise be hidden in $\hat{\theta}$ without hurting the prediction accuracy?

Example 3.9 (Spiked covariance). *Consider a linear regression setting where the number of features p is much larger than the number of samples n . Suppose the covariance matrix Σ of data has a Spiked structure with one giant eigenvalue, $\lambda_1 = 1$ (the signal direction), plus a long tail of tiny eigenvalues, $\lambda_2 = \dots = \lambda_p = \frac{1}{p}$ (the noise directions). As p goes to infinity, the effective rank of the data becomes very large. With the minimum-norm interpolator, on the other hand, the model uses the first dimension to capture the true signal, and the remaining $p - 1$ dimensions to absorb the noise level. Because the noise is spread over many dimensions, the amount of noise assigned to any single direction is negligible. As a result, the excess risk goes to zero even though the model perfectly interpolated noisy data.¹⁰*

The following result, by Bartlett et al. (2020), captures this example in more generality. There exists universal constants b, c such that the excess risk of the minimum norm interpolator satisfies:

$$R(\hat{\theta}) \leq c \left(\text{bias}(\theta^*, \Sigma, n) + \sigma^2 \left(\frac{k^*}{n} + \frac{n}{R_{k^*}(\Sigma)} \right) \right),$$

where $k^* = \min \{k \geq 0 : r_k(\Sigma) \geq bn\}$, and

$$r_k(\Sigma) := \frac{\sum_{i>k} \lambda_i}{\lambda_{k+1}}$$

$$R_k(\Sigma) := \frac{\left(\sum_{i>k} \lambda_i \right)^2}{\sum_{i>k} \lambda_i^2}.$$

The bias is given by

$$\text{bias}(\theta^*, \Sigma, n) = \|\theta_{k+1:\infty}^*\|_{\Sigma_{k+1:\infty}}^2 + \|\theta_{1:k}^*\|_{\Sigma_{1:k}^{-1}}^2 \left(\frac{\sum_{i>k} \lambda_i}{n} \right)^2$$

Recall that the benign overfitting prediction rule $\hat{f}$ decomposes as $\hat{f}_0 + \Delta$.

- $\hat{f}_0$ is the prediction component: k^* -dimensional subspace corresponding to $\lambda_1, \dots, \lambda_{k^*}$
- Δ is the benign overfitting component: orthogonal subspace. Δ is benign only if $R_{k^*} \gg n$

In other words, to avoid harming prediction accuracy, the noise energy must be distributed across many unimportant directions.

3.4 PAC-Bayes generalization bounds (Lectures 14)

Next, we present a PAC-Bayes generalization bound that depends on the trace of the Hessian. Our bound can be related to the notion of trace norm, which can be viewed as a measure of model complexity. For instance, in the context of matrix recovery, the trace norm refers

¹⁰See also the celebrated Marchenko–Pastur Law (and also Wigner’s Semicircle Law), and its applications in wireless communication (Tulino and Verdú, 2004). The result states that for a large N by N symmetric matrix where the entries are sampled independently and identically from a normal distribution with mean 0 and variance σ^2 , as $N \rightarrow \infty$, the density of the eigenvalues follows a semicircle:

$$f(x) = \frac{1}{2\pi R^2} \sqrt{4R^2 - x^2},$$

supported between $[-2\sigma\sqrt{N}, 2\sigma\sqrt{N}]$.

to the sum of the singular values of a matrix and links to the recovery quality of a matrix completion algorithm (Srebro and Shraibman, 2005).

To motivate this analysis, let's suppose that we have a pretrained model and we would like to fine-tune this model on a downstream task of interest. The weights of this pretrained model can be viewed as our prior belief of the target hypothesis in the PAC-Bayes analysis. Once we have learned a model through fine-tuning the pretrained model on the target task data, we can view this fine-tuned as our adjusted posterior in PAC-Bayes analysis.

More precisely, let $\mathcal{D} \subseteq \mathcal{X} \times \mathcal{Y}$ be an unknown data distribution, supported on the feature space $\mathcal{X}$ and the label space $\mathcal{Y}$. Given n random samples $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ drawn from $\mathcal{D}$, the empirical loss measured by loss function ℓ applied to a model f_W with $W \in \mathbb{R}^d$ is:

$$\hat{L}(W) = \frac{1}{n} \sum_{i=1}^n \ell(f_W(x_i), y_i).$$

The population loss is

$$L(W) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(f_W(x), y)].$$

It is sufficient to think that the empirical loss is less than the population loss, and the goal is to bound the gap between $\hat{L}(W)$ and $L(W)$ from above (Shalev-Shwartz and Ben-David, 2014). With this setting in mind, we now describe several PAC-Bayesian bounds, which incorporate the prior into the analysis without assuming that the prior is “correct.”

In Occam's bound, we assume a countable hypothesis class, and the algorithm outputs a single hypothesis. More specifically, let $\mathcal{H}$ be a countable hypothesis class. Let the loss function be bounded $\ell(z, h) \leq [0, 1]$. Let $\mathcal{P}$ be any prior distribution over $\mathcal{H}$. Then with probability at least $1 - \delta$, we have that

$$\forall h \in \mathcal{H}, L(h) \leq \hat{L}(h) + \sqrt{\frac{\log\left(\frac{1}{\mathcal{P}(h)}\right) + \log(\delta^{-1})}{2n}}.$$

Proof. By Hoeffding's inequality, we have that for any fixed $h \in \mathcal{H}$:

$$\Pr[L(h) \geq \hat{L}(h) + \epsilon] \leq \exp(-2n\epsilon^2).$$

If we set the RHS to $\delta\mathcal{P}(h)$, then we get that with probability at least $\delta\mathcal{P}(h)$,

$$L(h) \geq \hat{L}(h) + \sqrt{\frac{\log\left(\frac{1}{\delta\mathcal{P}(h)}\right)}{2n}}.$$

Applying the union bound across all of $h \in \mathcal{H}$, we have that with probability at most δ , there exists an $h \in \mathcal{H}$ such that the above is true. By taking the contra-positive, we have derived Occam's bound.

There are two things lacking about the Occam bound. It only applies to countable $\mathcal{H}$, which does not include the set of all weight vectors, for example. It only embraces half of the Bayesian story. While we have a prior $\mathcal{P}(h)$, only a single $h \in \mathcal{H}$ is returned rather than a full posterior $\mathcal{Q}(h)$.

Next, we show the PAC-Bayesian theorem (McAllester, 1998), which fixes the above problem. Let the loss function be bounded between 0 and C , for some fixed constant $C > 0$.

Let $\mathcal{P}$ be any prior distribution over $\mathcal{H}$. Let $\mathcal{Q}$ be any posterior distribution over $\mathcal{H}$, which is a function of the training data. Then with probability at least $1 - \delta$,

$$\mathbb{E}_{h \sim \mathcal{Q}} [L(h)] \leq \mathbb{E}_{h \in \mathcal{Q}} [\hat{L}(h)] + C \sqrt{\frac{D_{KL}(\mathcal{Q}||\mathcal{P}) + \log(2n\delta^{-1})}{2(n-1)}}, \quad (55)$$

which holds for all posterior $\mathcal{Q}$.

Example 3.10 (Supervised fine-tuning of language models). *In the case of SFT, one can set both $\mathcal{P}$ and $\mathcal{Q}$ as Gaussian distribution with variance σ^2 , centered at the pretrained and fine-tuned models, respectively. The perturbations provide an evaluation of the stability of the loss surface, under perturbations of noise injection.*

A more extreme version of model attack is jailbreaking (Chao et al., 2025). The problem of jailbreaking can be formulated as follows. Let $P = x_{1:n}$ be a sequence of n tokens, where $x_{1:n} = (x_1, \dots, x_n)$. Let the response $R = x_{n+1:n+L}$ be a sequence of L tokens. Assume there is a generative model that outputs the response:

$$q_T^*(x_{n+1:n+L} \mid x_{1:n}) = \prod_{i=1}^L q_T(x_{n+i} \mid x_{1:n+i-1}),$$

where $q_T : V^* \rightarrow \Delta(V)$ for some vocabulary V . The problem of jailbreaking involves finding a prompt sequence P such that $\text{JUDGE}(P, R) = 1$, for $R \sim q_T(P)$, where $\text{JUDGE} : V^* \times V^* \rightarrow \{0, 1\}$ is a binary-valued function that determines whether a prompt-response pair (P, R) is jailbroken.

Definition 3.11 (KL divergence). *For two distributions $\mathcal{P}$ and $\mathcal{Q}$, the KL divergence between $\mathcal{P}$ and $\mathcal{Q}$ is given by:*

$$D_{KL}(\mathcal{P}||\mathcal{Q}) := \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx. \quad (56)$$

Example 3.12 (KL divergence between two Gaussian distributions). *Consider two (scaled) isotropic Gaussian distributions with different centers $\mu_1 \in \mathbb{R}^d, \mu_2 \in \mathbb{R}^d$: $\mathcal{P} = \mathcal{N}(\mu_1, \Sigma)$ and $\mathcal{Q} = \mathcal{N}(\mu_2, \Sigma)$, where $\Sigma = \sigma^2 \text{Id}$.*

The KL divergence, according to equation (56), is given by:

$$\begin{aligned} & \int_{\mathbb{R}^d} \frac{\exp \left(-\frac{1}{2}(\mathbf{x} - \mu_1)^\top \Sigma (\mathbf{x} - \mu_1) \right)}{\sqrt{(2\pi)^d \det(\Sigma)}} \left(-\frac{\sigma^2}{2} \left(2\langle \mu_2 - \mu_1, \mathbf{x} \rangle + \|\mu_1\|^2 - \|\mu_2\|^2 \right) \right) d\mathbf{x} \\ &= -\frac{\sigma^2}{2} \left(\|\mu_1\|^2 - \|\mu_2\|^2 + 2 \mathbb{E} [\langle \mu_2 - \mu_1, \mathbf{x} \rangle] \right) \\ &= -\frac{\sigma^2}{2} \left(\|\mu_1\|^2 - \|\mu_2\|^2 + 2\langle \mu_2 - \mu_1, \mu_1 \rangle \right) = \frac{\sigma^2}{2} \|\mu_1 - \mu_2\|^2. \end{aligned} \quad (57)$$

In summary, the KL divergence gives a distance measure between two distributions, and one can verify that it satisfies the triangle inequality required by a distance metric.

Proof. The first step is to derive the so-called Donsker-Varadhan change-of-measure result. For a prior $\mathcal{P}$, the change of measure inequality states that for any measure $\mathcal{Q}$, any function $\phi : \mathcal{H} \rightarrow \mathbb{R}$,

$$\mathbb{E}_{f \sim \mathcal{Q}} [\phi(f)] \leq \log \mathbb{E}_{f \in \mathcal{P}} [\exp(\phi(f))] + D_{KL}(\mathcal{Q}||\mathcal{P}). \quad (58)$$

To show equation (58), let us define a distribution $\mathcal{P}_G$ via its density with respect to $\mathcal{P}$ as:

$$\frac{d\mathcal{P}_G}{d\mathcal{P}}(f) = \frac{\exp(\phi(f))}{\mathbb{E}_{h \sim \mathcal{P}} [\exp(\phi(h))]}.$$

This is called the Gibbs measure. Notice that

$$\begin{aligned} D_{KL}(\mathcal{Q}||\mathcal{P}) - D_{KL}(\mathcal{Q}||\mathcal{P}_G) &= \int \left(\log \left(\frac{d\mathcal{Q}}{d\mathcal{P}} \right) - \log \left(\frac{d\mathcal{Q}}{d\mathcal{P}_G} \right) \right) d\mathcal{Q} \\ &= \int \log \left(\frac{d\mathcal{P}_G}{d\mathcal{P}} \right) d\mathcal{Q} \\ &= \mathbb{E}_{f \sim \mathcal{Q}} \left[\log \left(\frac{\exp(\phi(f))}{\mathbb{E}_{h \sim \mathcal{P}} [\exp(\phi(h))]} \right) \right] \\ &= \mathbb{E}_{f \sim \mathcal{Q}} [\phi(f)] - \log \mathbb{E}_{h \sim \mathcal{P}} [\exp(\phi(h))]. \end{aligned}$$

Thus,

$$\begin{aligned} \log \mathbb{E}_{h \sim \mathcal{P}} [\exp(\phi(h))] &= \mathbb{E}_{f \sim \mathcal{Q}} [\phi(f)] - D_{KL}(\mathcal{Q}||\mathcal{P}) + D_{KL}(\mathcal{Q}||\mathcal{P}_G) \\ &\geq \mathbb{E}_{f \sim \mathcal{Q}} [\phi(f)] - D_{KL}(\mathcal{Q}||\mathcal{P}), \end{aligned}$$

since the KL divergence between two distributions is always non-negative. Notice that the inequality holds if and only if $\mathcal{Q} = \mathcal{P}_G$.

Next, in order to prove equation (55), we consider the function $2(n-1)(\Delta(f))^2$ as ϕ , where $\Delta(f) = L(f) - \hat{L}_n(f)$. For a fixed f , we can bound $\Delta(f)$ using Hoeffding's inequality as:

$$\Pr[|\Delta(f)| \geq \epsilon] \leq 2 \exp(-2n\epsilon^2), \quad (59)$$

which holds for any positive $\epsilon < 1$. The second step is to turn equation (59) into a high-probability bound. We apply the Donsker-Varadhan change-of-measure, and get

$$\mathbb{E}_{f \sim \mathcal{P}} [\exp(2(n-1)(\Delta(f))^2)] \geq \exp \left(2(n-1) \mathbb{E}_{f \sim \mathcal{Q}} [(\Delta(f))^2] - D_{KL}(\mathcal{Q}||\mathcal{P}) \right).$$

Let us take the expectation over the training sample set S (recall that $\mathcal{Q}$ depends on the training samples) on both sides and notice that the prior $\mathcal{P}$ does not depend on the training samples. We get

$$\mathbb{E}_S \left[\mathbb{E}_{h \sim \mathcal{P}} [\exp(2(n-1)(\Delta(h))^2)] \right] \geq \mathbb{E}_S \left[\exp \left(2(n-1) \mathbb{E}_{f \sim \mathcal{Q}} [(\Delta(f))^2] - D_{KL}(\mathcal{Q}||\mathcal{P}) \right) \right]$$

Therefore, we have

$$\begin{aligned} \mathbb{E}_S [\exp(2(n-1)(\Delta(f))^2)] &= \int_0^\infty \Pr [\exp(2(n-1)(\Delta(f))^2) \geq s] ds \\ &\leq 1 + \int_1^\infty \Pr \left[|\Delta(f)| \geq \sqrt{\frac{\log(s)}{2(n-1)}} \right] ds \\ &\leq 1 + \int_1^\infty 2 \exp \left(-2n \cdot \frac{\log(s)}{2(n-1)} \right) ds \\ &= 1 + 2(n-1) \leq 2n \end{aligned}$$

As for the last step, we notice that

$$-\int_1^\infty s^{-1-\frac{1}{n-1}} ds = -\int_1^\infty d\left((n-1)s^{-\frac{1}{n-1}}\right) = n-1$$

Then, by Markov's inequality,

$$\Pr \left[\exp \left(2(n-1) \mathbb{E}_{f \sim \mathcal{Q}} [(\Delta(f))^2] - D_{KL}(\mathcal{Q}||\mathcal{P}) \right) \geq \frac{2n}{\delta} \right] \leq \delta$$

That is, with probability at least $1 - \delta$,

$$2(n-1) \mathbb{E}_{f \sim \mathcal{Q}} [(\Delta(f))^2] \leq \log \left(\frac{2n}{\delta} \right) + D_{KL}(\mathcal{Q}||\mathcal{P})$$

Using Jensen's inequality, the left is greater than

$$2(n-1) \left(\mathbb{E}_{f \sim \mathcal{Q}} [\Delta(f)] \right)^2$$

Taking square root on both sides gives us the McAllester's bound.

Example 3.13 (Shifted Gaussians). *Let $\mathcal{P} = \mathcal{N}(0, 1)$, $\mathcal{Q} = \mathcal{N}(\mu, 1)$. Consider the test function $f(x) = tx$ for some real value t . The DV inequality (58) states that*

$$\mathbb{E}_{\mathcal{Q}} [tx] \leq D_{KL}(\mathcal{Q}||\mathcal{P}) + \log \mathbb{E}_{\mathcal{P}} [e^{tx}]$$

To compute each term, we note that the left-hand side is $t\mu$. For the right hand side, by Example 3.12, the KL divergence term is $\mu^2/2$. The second term is $e^{t^2/2}$ based on the moment-generating function of a standard Gaussian. Putting together, (58) is equivalent to $2t\mu \leq \mu^2 + t^2$, which recovers a simple quadratic bound.

As a remark, the equality holds when $\mathcal{Q} = \mathcal{P}_G$. In the above example, when $t = \mu$, or when $\mathcal{Q} = \mathcal{N}(t, 1)$.

Another version of the PAC-Bayesian theorem is shown by Catoni, 2007 (see also Theorem 3.5, Alquier, 2021). Suppose the loss function $\ell(f_W(x), y)$ lies in a bounded range $[0, C]$ given any $x \in \mathcal{X}$ with label y . For any $\beta \in (0, 1)$ and $\delta \in (0, 1)$, with probability at least $1 - \delta$, the following holds:

$$L_{\mathcal{Q}}(W) \leq \frac{1}{\beta} \hat{L}_{\mathcal{Q}}(W) + \frac{C(D_{KL}(\mathcal{Q}||\mathcal{P}) + \log \frac{2\sqrt{n}}{\delta})}{2\beta(1-\beta)n}, \quad (60)$$

where $\mathcal{P}$ refers to the prior distribution, C refers to the upper bound on the loss value ℓ . For analyzing fine-tuning, we view $\mathcal{P}$ as centered at the pretrained model, with covariance matrix $\sigma^2 \text{Id}$.

Remark 3.14 (Equation (60) generalizes equation (55)). *Let us now illustrate that equation (60) in fact generalizes the earlier equation (55). To see this, we can treat the expression in equation (60) as a function in the form of $f(\beta) = \frac{a}{\beta} + \frac{b}{\beta(1-\beta)}$. One can verify that the minimum of this is achieved when $\beta^* = \frac{\sqrt{a+b}}{\sqrt{a+b}+\sqrt{b}}$, leading to $f(\beta^*) = a + 2b + 2\sqrt{(a+b)b}$, or*

$$\hat{L}_{\mathcal{Q}}(W) + O(n^{-1}) + 2\sqrt{\frac{C(C + O(n^{-1}))(D_{KL}(\mathcal{P}||\mathcal{Q}) + \log(2n\delta^{-1}))}{n}},$$

which leads to equation (55) for some large enough range of n .

Proof of equation (60). The proof sketch of equation (60) is similar to equation (55), which leverages the Donsker-Varadyan change-of-measure. The difference is in how we approach step (59). First, we still have equation (58), which holds for any $\mathcal{Q}$ and function ϕ .

Next, one instead show an exponential moment bound for one fixed hypothesis W as

$$\mathbb{E} \left[\exp(n D_{KL}(\hat{L}_{\mathcal{Q}} \| L_{\mathcal{Q}})) \right] \leq 2\sqrt{n}.$$

Next, let $\phi(f) = n(D_{KL}(\mathcal{Q} \| \mathcal{P}))$. We apply the Donsker-Varadyan change-of-measure (58) to ϕ . Then, we turn the above into a high-probability bound. Using Markov's inequality, with probability $1 - \delta$ over the sample S , we get a result such that

$$\mathbb{E}_{\mathcal{Q}} \left[D_{KL}(\hat{L}(W) \| L(W)) \right] \leq \frac{D_{KL}(\mathcal{Q} \| \mathcal{P}) + \log(2\sqrt{n}/\delta)}{n}.$$

Using convexity of the KL operator, we get

$$D_{KL}(\hat{L}_{\mathcal{Q}}(W) \| L_{\mathcal{Q}}(W)) \leq \frac{D_{KL}(\mathcal{Q} \| \mathcal{P}) + \log(2\sqrt{n}/\delta)}{n}.$$

Lastly, we use the following fact

$$D_{KL}(q \| p) \leq \varepsilon \implies p \leq \frac{q}{\beta} + \frac{\varepsilon}{2\beta(1-\beta)}, \quad \forall \beta \in (0, 1),$$

which completes the proof of equation (60).

3.5 Noise-sensitivity generalization bounds (Lecture 15)

Let W be any learned hypothesis within the hypothesis space, denoted as $\mathcal{H}$. This generalization bound will apply uniformly to W within the hypothesis space. We state this result, including the required assumptions, as follows. Assume that the loss function ℓ is bounded between 0 and C for a fixed constant $C > 0$ on the data distribution $\mathcal{D}$. Suppose $\ell(f_W(\cdot), \cdot)$ is twice-differentiable in W and the Hessian matrix $\nabla^2[\ell(f_W(\cdot), \cdot)]$ is Lipschitz continuous within the hypothesis space. Suppose for any W in $\mathcal{H}$, the trace norm of the Hessian is less than α :

$$\alpha := \max_{W \in \mathcal{H}} \max_{(x,y) \sim \mathcal{D}} \text{Tr} \left[\nabla^2 \ell(f_W(x), y) \right], \quad (61)$$

and the ℓ_2 -norm of W is at most r for any $W \in \mathcal{H}$. Then, for any W in $\mathcal{H}$, with probability at least $1 - \delta$ for any $\delta > 0$, the following must hold, for any ϵ close to zero:

$$L(W) \leq (1 + \epsilon) \hat{L}(W) + (1 + \epsilon) \sqrt{\frac{C\alpha r^2}{n}} + O\left(n^{-\frac{3}{4}} \log(\delta^{-1})\right). \quad (62)$$

An important takeaway of this bound is that we could now measure the RHS in practice, and this leads to a non-vacuous bound on the generalization error. We experiment with seven methods, including fine-tuning with and without early stopping, distance-based regularization, label smoothing, mixup, etc. Figure 5 shows that the Hessian measure (i.e., the second part of equation (62)) correlates with the generalization errors of these methods. Second, we plot the Hessian measure (averaged over all layers) between fine-tuning with implicit (early stopping) regularization and explicit (distance-based) regularization. We find that the Hessian measure is smaller for explicit regularization than implicit regularization.

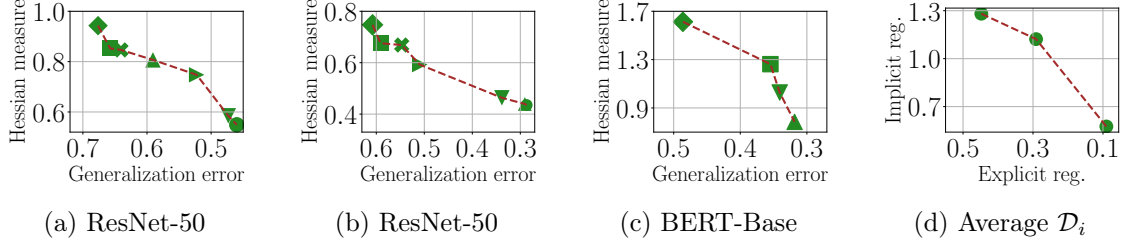


Figure 5: 5a, 5b, 5c: The Hessian measures accurately correlate with empirical generalization errors for seven fine-tuning methods. 5d: $\mathcal{D}_i$ is smaller for fine-tuning with explicit distance-based regularization than implicit early-stopped regularization. Each dot represents the generalization error and the Hessian measures for a single fine-tuned model using a specific regularization method.

Proof. We provide a high-level illustration of the proof of equation (61). Let $\mathcal{Q}$ denote the *posterior* distribution. Specifically, we consider $\mathcal{Q}$ as being centered at the learned hypothesis W (which could be anywhere within the hypothesis space), given by a Gaussian distribution $\mathcal{N}(W, \sigma^2 \text{Id}_p)$, where Id_p denotes the p by p identity matrix. Given a sample $U \sim \mathcal{N}(0, \sigma^2 \text{Id}_p)$, let the perturbed loss be given by

$$\ell_{\mathcal{Q}}(f_W(x), y) = \mathbb{E}_U [\ell(f_{W+U}(x), y)]. \quad (63)$$

Then, let $\hat{L}_{\mathcal{Q}}(W)$ be the averaged value of $\ell_{\mathcal{Q}}(f_W(\cdot), \cdot)$, taken over n empirical samples from the training dataset. Likewise, let $L_{\mathcal{Q}}(W)$ be the population average of $\ell_{\mathcal{Q}}(f_W(\cdot), \cdot)$, in expectation over an unseen data sample from the underlying data distribution.

By Taylor’s expansion of $\ell_{\mathcal{Q}}$, we show that:

$$L_{\mathcal{Q}}(W) = L(W) + \frac{\sigma^2}{2} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\text{Tr} [\nabla^2 \ell(f_W(x), y)]] + O(\sigma^3) \quad (64)$$

$$\hat{L}_{\mathcal{Q}}(W) = \hat{L}(W) + \frac{\sigma^2}{2n} \sum_{i=1}^n \text{Tr} [\nabla^2 \ell(f_W(x_i), y_i)] + O(\sigma^3). \quad (65)$$

Since the Hessian operator is Lipschitz continuous by assumption, we can bound the gap between the above two quantities with ϵ -covering arguments. By plugging in these results back to the PAC-Bayes bound of equation (60), after some calculation, we can get:

$$L(W) \leq \frac{1}{\beta} \hat{L}(W) + \frac{\sigma^2(1-\beta)\alpha}{2\beta} + \frac{Cr^2/2\sigma^2}{2\beta(1-\beta)n} + O\left(\sigma^3 + \frac{\sigma^2\sqrt{p}}{\sqrt{n}} + \frac{\log(\delta^{-1})}{n}\right). \quad (66)$$

In particular, the above uses the fact that the ℓ_2 -norm of W is less than r for any $W \in \mathcal{H}$. The KL divergence is discussed below. Suppose $\mathcal{P} = N(X, \Sigma)$ and $\mathcal{Q} = N(Y, \Sigma)$ are both Gaussian distributions with mean vectors given by $X \in \mathbb{R}^p, Y \in \mathbb{R}^p$, and population covariance matrix $\Sigma \in \mathbb{R}^{p \times p}$. The KL divergence between $\mathcal{P}$ and $\mathcal{Q}$ is equal to

$$KL(\mathcal{Q}||\mathcal{P}) = \frac{1}{2}(X - Y)^{\top} \Sigma^{-1}(X - Y).$$

Specifically, if $\Sigma = \sigma^2 \text{Id}_p$, then the above simplifies to

$$KL(\mathcal{Q}||\mathcal{P}) = \frac{\|X - Y\|^2}{2\sigma^2}.$$

By choosing σ^2 and β to minimize equation (66), we will obtain equation (62). This summarizes the high-level proof idea. For a complete proof, please refer to the work of Zhang et al. (2024).

4 Statistical modeling of representation learning

In many modern ML settings, one would like to use a large, pretrained model, to solve a downstream task. Instead of training from scratch, these models provide a representation of the input that are then used to make predictions on the input data. While this paradigm has been quite useful in practice, practitioners have also observed problematic outcomes. For instance, negative transfer refers to a situation where transfer learning from a source task to predict a target task negatively hurts the prediction performance, as compared to learning using the target task data alone. However, exactly how and why negative transfer occurs has not been fully resolved — Next, we provide a rigorous study of this phenomenon in the setting of transfer learning linear regression.

4.1 Transfer learning linear regression (Lecture 15)

We first introduce the problem setup. Suppose there are n_1 samples drawn from a p -dimensional distribution D_1 with real-valued labels in a *source* task. Suppose there are n_2 samples drawn from a p -dimensional distribution D_2 with real-valued labels in the *target* task. Let $x_1^{(i)}, x_2^{(i)}, \dots, x_{n_i}^{(i)}$ denote feature covariates. Let $y_1^{(i)}, y_2^{(i)}, \dots, y_{n_i}^{(i)}$ be the prediction labels. We assume a linear model specified by an unknown parameter vector $\beta^{(i)} \in \mathbb{R}^p$:

$$y_j^{(i)} = (x_j^{(i)})^\top \beta^{(i)} + \varepsilon_j^{(i)}, \quad \text{for any } j = 1, \dots, n_i,$$

where $\varepsilon_j^{(i)} \in \mathbb{R}$ denotes a random noise variable with mean zero and variance σ^2 . We refer to the first task as the *source* task and the second task as the *target* task for ease of presentation. We assume $n_2 > p$ while n_1 can be either less or greater than p .

In linear regression, there is typically a distinction between fixed designs vs. random designs. Fixed designs refer to a situation where the covariates are deterministic, e.g., think of the population demographic data for one example. By contrast, random designs correspond to cases where the covariates may involve observational uncertainties. For this section, we shall study random designs where the covariates are given by

$$x_j^{(i)} = (\Sigma^{(i)})^{\frac{1}{2}} z_j^{(i)}, \quad \text{for } i \in \{1, 2\} \text{ and for any } j = 1, \dots, n_i,$$

where $z_j^{(i)}$ consists of independent and identically distributed entries of mean zero and variance one, and $\Sigma^{(1)} \in \mathbb{R}^{p \times p}$ and $\Sigma^{(2)} \in \mathbb{R}^{p \times p}$ denote the population covariance matrices. Let $X^{(i)} = (x_1^{(i)}, \dots, x_{n_i}^{(i)})^\top \in \mathbb{R}^{n_i \times p}$ denote a matrix that corresponds to all the covariates of task i , for $i \in \{1, 2\}$.

Let $Y^{(1)} \in \mathbb{R}^{n_1}$, $Y^{(2)} \in \mathbb{R}^{n_2}$ be the vectors of labels. Then, consider the following objective, parameterized by a shared variable $B \in \mathbb{R}^p$:

$$\ell(B) = \|X^{(1)}B - Y^{(1)}\|^2 + \|X^{(2)}B - Y^{(2)}\|^2. \quad (67)$$

Notice that B provides a shared feature vector for both tasks. This way of estimation is consistent with the hard parameter sharing (HPS) architecture that has been used in the earlier literature (Caruana, 1997). Thus, we shall call the minimizer of $\ell(B)$ the HPS estimator, denoted as $\hat{\beta}_2^{\text{HPS}}$ for short.

4.1.1 Bias and variance

The risk of $\hat{\beta}_i^{\text{HPS}}$ over an unseen sample (x, y) of the target task i is given by (under the mean squared loss):

$$\mathbb{E}_{x,y} \left[\left\| y - x^\top \hat{\beta}_i^{\text{HPS}} \right\|^2 \right] = \left\| \Sigma^{(i)\frac{1}{2}} \left(\hat{\beta}_i^{\text{HPS}} - \beta^{(i)} \right) \right\|^2 + \sigma^2.$$

The excess risk is the difference between the above risk and the expected risk of the population risk optimizer:

$$L(\hat{\beta}_i^{\text{HPS}}) := \left\| \Sigma^{(i)\frac{1}{2}} \left(\hat{\beta}_i^{\text{HPS}} - \beta^{(i)} \right) \right\|^2. \quad (68)$$

Next, we present a bias-variance decomposition of the excess risk of HPS. Denote the sum of both tasks' sample covariance matrix as:

$$\hat{\Sigma} = X^{(1)\top} X^{(1)} + X^{(2)\top} X^{(2)}. \quad (69)$$

The bias and variance formulas are defined as

$$L_{\text{bias}} = \left\| \Sigma^{(2)\frac{1}{2}} \hat{\Sigma}^{-1} (X^{(1)\top} X^{(1)}) (\beta^{(1)} - \beta^{(2)}) \right\|^2, \quad (70)$$

$$L_{\text{var}} = \sigma^2 \text{Tr} [\Sigma^{(2)} \hat{\Sigma}^{-1}]. \quad (71)$$

The proof of the above is based on algebraic calculations. By minimizing $\ell(B)$ in (68), we obtain the minimizer as

$$\hat{\beta}_2^{\text{HPS}} = \left((X^{(1)})^\top X^{(1)} + (X^{(2)})^\top X^{(2)} \right)^{-1} \left((X^{(1)})^\top Y^{(1)} + (X^{(2)})^\top Y^{(2)} \right)$$

By simplifying the above a little bit, we get

$$\begin{aligned} \hat{\beta}_2^{\text{HPS}} &= \beta^{(2)} + \left((X^{(1)})^\top X^{(1)} + (X^{(2)})^\top X^{(2)} \right)^{-1} \left((X^{(1)})^\top \varepsilon^{(1)} + (X^{(2)})^\top \varepsilon^{(2)} \right) \\ &\quad + \left((X^{(1)})^\top X^{(1)} + (X^{(2)})^\top X^{(2)} \right)^{-1} (X^{(1)})^\top X^{(1)} (\beta^{(1)} - \beta^{(2)}). \end{aligned}$$

Plugging this into the excess risk, we get that in expectation over the randomness of $\varepsilon^{(1)}$ and $\varepsilon^{(2)}$,

$$\mathbb{E} [L(\hat{\beta}_2^{\text{HPS}})] = L_{\text{bias}} + L_{\text{var}}.$$

We can now compare the above bias-variance to that of single-task learning on the target task. That would just be

$$\sigma^2 \text{Tr} [\Sigma^{(2)} ((X^{(2)})^\top X^{(2)})^{-1}],$$

which is the variance of the ordinary least squares (OLS) estimator for the target task. The bias and variance decomposition above allows us to reason about transfer effects by comparing the bias and variance of HPS with that of OLS. The bias of HPS is always larger than that of OLS. On the other hand, the variance of HPS is always lower than that of OLS. By Woodbury matrix identity, the variance of OLS is always higher than formula (71):

$$\text{Tr} [\Sigma^{(2)} (X^{(2)\top} X^{(2)})^{-1}] \geq \text{Tr} [\Sigma^{(2)} \hat{\Sigma}^{-1}]. \quad (72)$$

Thus, the transfer effect of HPS is determined by the bias-variance decomposition: the bias always increases while the variance always decreases. Whether or not the transfer effect of HPS is positive depends on which effect dominates.

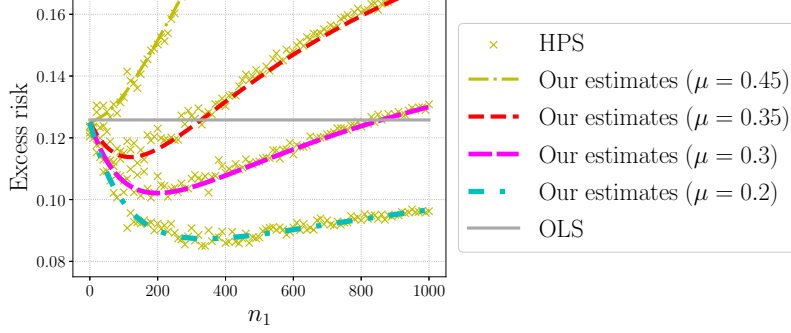


Figure 6: We illustrate the performance of HPS vs. OLS (ordinary least squares) for different levels of model shifts, indicated by $\mu \approx \|\beta^{(1)} - \beta^{(2)}\|/\sqrt{2}$. We consider two linear regression tasks and vary the sample size of the source task, denoted as n_1 , and μ , while holding the target task as fixed. The gray line refers to the performance of OLS. Any point above the gray line represents negative transfer, while any point below represents positive transfer. In this simulation, we set $p = 100$, $n_2 = 300$, and $\sigma^2 = \frac{1}{4}$. We sample the covariates X from a p -dimensional isotropic Gaussian and plot the excess risk of HPS (see equation (68)).

4.1.2 Numerical example

We will consider a random-effects model. Each $\beta^{(i)}$ consists of two components, in this case, one shared by all tasks and one that is task-specific. Let β_0 be the shared component and γ_i be the i -th task-specific component. For any i , the i -th model vector is equal to $\beta^{(i)} = \beta_0 + \gamma_i$. The entries of the task-specific component γ_i are drawn independently from a Gaussian distribution with mean zero and variance $p^{-1}\mu^2$, for a parameter $\mu > 0$. In expectation, the Euclidean distance between the two model vectors is equal to $2\mu^2$.

Then, we have the following phase transition in the random-effects model. With high probability over the randomness of training samples and model vectors.

1. If $\mu^2 \frac{\text{Tr}[\Sigma^{(1)}]}{p} \leq \frac{\sigma^2 p}{2(n_2 - p)}$, then the transfer effect is always positive:

$$L(\hat{\beta}_2^{\text{HPS}}) \leq L(\hat{\beta}_2^{\text{OLS}}). \quad (73)$$

2. If $\frac{\sigma^2 p}{2(n_2 - p)} < \mu^2 \frac{\text{Tr}[\Sigma^{(1)}]}{p} < \frac{\sigma^2 n_2}{2(n_2 - p)}$, then there exists a deterministic value $n_0 > 0$ (which may not be an integer) such that if $n_1 \leq n_0$, then equation (73) holds; otherwise if $n_1 > n_0$, then

$$L(\hat{\beta}_2^{\text{OLS}}) \leq L(\hat{\beta}_2^{\text{HPS}}). \quad (74)$$

3. If $\mu^2 \frac{\text{Tr}[\Sigma^{(1)}]}{p} \geq \frac{\sigma^2 n_2}{2(n_2 - p)}$, then equation (74) holds for any $n_1 > p$.

Figure 6 demonstrates an illustration of this tradeoff.

4.1.3 Minimax lower bounds

The above estimator seems like a natural choice in practice. However, how would we know if there won't be another estimator that is much better than the above?

Example 4.1 (Estimating the mean of a Gaussian). Suppose you observe $X_1, \dots, X_n$ from $\mathcal{N}(\theta, 1)$, where $\theta \in \mathbb{R}$ is unknown, and you want to estimate θ under squared error loss $(\hat{\theta} - \theta)^2$.

The minimax risk is given by

$$\inf_{\hat{\theta}} \sup_{\theta \in \mathbb{R}} \mathbb{E}_{\theta} [(\hat{\theta} - \theta)^2]$$

A minimax lower bound shows that no estimator can make this risk arbitrarily small. In fact, one can show that the above is at least $\frac{1}{n}$. In particular, the sample mean achieves risk $1/n$ (due to variance), matching the minimax rate.

In the context of transfer learning linear regression, suppose we are trying to estimate an unknown parameter denoted as $\beta^{(2)}$. We are given n_2 samples drawn from a linear parametric model, following $\beta^{(2)}$, with isotropic Gaussian covariates $X^{(2)}$, contaminated by Gaussian noise with variance σ^2 . Then, we are given n_1 samples from another linear parametric model, following $\beta^{(1)}$, again with isotropic Gaussian covariates $X^{(1)}$, contaminated by Gaussian noise with variance σ^2 . The parameter vectors belong to the set

$$\Theta(\mu) = \left\{ \beta^{(1)} \in \mathbb{R}^p, \beta^{(2)} \in \mathbb{R}^p : \|\beta^{(1)} - \beta^{(2)}\| \leq \mu, \|\beta^{(1)}\| \leq 1, \|\beta^{(2)}\| \leq 1 \right\}.$$

Let $\hat{\beta}$ be any estimation procedure that, given the above $n_1 + n_2$ samples, produces an estimate of the unknown vector $\beta^{(2)}$. We prove the following minimax rate on the estimation error of $\hat{\beta}$. Assume that $n_1 \geq (1 + \tau)p$ and $n_2 \geq (1 + \tau)p$ for a constant $\tau > 0$. For any $(\beta^{(1)}, \beta^{(2)})$ within the set $\Theta(\mu)$, we have that

$$\inf_{\hat{\beta}} \sup_{\Theta(\mu)} \mathbb{E} \left[\frac{1}{n_2} \left\| \tilde{X}^{(2)} \left(\hat{\beta} - \beta^{(2)} \right) \right\|^2 \right] \geq c \left(\min \left(\frac{n_1^2 \mu^2}{(n_1 + n_2)^2}, \frac{\sigma^2 p}{n_2} \right) + \frac{\sigma^2 p}{n_1 + n_2} \right), \quad (75)$$

where the expectation is over the randomness of $X^{(1)}, X^{(2)}, Y^{(1)}, Y^{(2)}$ and an independently drawn $\tilde{X}^{(2)}$ that follows the same distribution as $X^{(2)}$. c is a fixed constant that does not grow with p .

The lower bound in the right-hand side of equation (75) involves two parts:

- For the first part, if μ is large, the source samples are not helpful, so the rate $\frac{\sigma^2 p}{n_2}$ is the OLS rate using only the target task samples. If μ is small and n_1 is large, then the rate μ^2 appears if we use the OLS estimator for the source task.
- For the second part, $\frac{\sigma^2 p}{n_1 + n_2}$ is the rate in the case without model drift, i.e., $\beta^{(1)} = \beta^{(2)}$.

This result provides a way to quantify the optimality of a statistical estimation algorithm. It turns out that the naive estimator described above is nearly optimal, with one caveat. When μ is very high, then we essentially throw away the source task data. Otherwise when μ is very small, then we combine source and target task data as HPS. It can be shown that this adjusted HPS estimator now matches the minimax lower bound.

The proof of the minimax lower bound is via a covering argument. In particular, we design a cover of the parameter vectors, taking into account the effect of model shifts (Wainwright, 2019).

4.2 Multitask and meta-learning (Lecture 15)

In multitask learning, one is concerned with k data distributions, $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_k$. The goal is to learn a predictor (or a small number of predictors) that performs well on all the k distributions simultaneously.

Meta-learning is another form of learning where you learn a meta-model during the training stage. At test time, you want to quickly adapt the meta-model to new samples.

Few-shot learning is a setting where a model must learn or adapt from only a small number of examples. For example, suppose you want a model to recognize a new class, like a rare bird species, but you only have 5 labeled images of it. If the model can still do well, this is called few-shot learning.

Many recent papers have sought to theoretically formalize the above learning paradigms. See, e.g., (Finn et al., [2019](#); Du et al., [2020](#); Wu et al., [2020](#)).

5 Optimization and generalization in reinforcement learning

The lecture note in this section draws mostly from Agarwal et al. (2019). In reinforcement learning, the interactions between the agent and the environment are often described by an infinite horizon, discounted Markov decision process (MDP), specified by: A state space $\mathcal{S}$, which may be finite or countably infinite; An action space $\mathcal{A}$, which may also be discrete or countably infinite; A transition model $P : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$, where $\Delta(\mathcal{S})$ is the space of probability distributions over $\mathcal{S}$ (i.e., the probability simplex); $\Pr(s'|s, a)$ is the probability of transitioning into state s' upon taking action a in state s . We use $P_{s,a}$ to denote the vector $\Pr(\cdot|s, a)$.

A reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, $r(s, a)$, denotes the (immediate) reward associated with taking action a in state s . More generally, $r(s, a)$ could be a random variable where the distribution depends on s, a . A discount factor $\gamma \in [0, 1)$ defines the (expected) horizon for the problem.

Finally, we assume an initial state distribution $\mu \in \Delta(\mathcal{S})$, which specifies how the initial state s_0 is generated.

Given an MDP $M = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \mu)$, the agent interacts with the environment according to the following protocol. The agent starts at some state $s_0 \sim \mu$; at each time step $t = 0, 1, 2, \dots$, the agent takes an action $a_t \in \mathcal{A}$, obtains the immediate reward $r_t = r(s_t, a_t)$, and observes the next state s_{t+1} sampled according to $s_{t+1} \sim \Pr(\cdot|s_t, a_t)$. The interaction record at time t ,

$$r_t = (s_0, a_0, r_0, s_1, \dots, s_t, a_t, r_t)$$

is called a trajectory, which includes the observed state at time t .

In the most general setting, a policy specifies a decision-making strategy in which the agent chooses actions adaptively based on the history of observations. Precisely, a policy is a mapping from a trajectory to an action, i.e., $\pi : \mathcal{H} \rightarrow \Delta(\mathcal{A})$ where $\mathcal{H}$ is the set of all possible trajectories (of all lengths) and $\Delta(\mathcal{A})$ is the space of probability distributions over $\mathcal{A}$.

We now define the value function for a given policy. For a fixed policy and a starting state $s_0 = s$, we define the value function $V^\pi : \mathcal{S} \rightarrow \mathbb{R}$ as the discounted sum of future rewards

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | \pi, s_0 = s \right]$$

where expectation is with respect to the randomness of the trajectory, that is, randomness in state transitions and stochasticity of π .

Similarly, the action-value (or Q -value) function $Q^\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is defined as

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | \pi, s_0 = s, a_0 = a \right]$$

Given a state s , the goal of the agent is to find a (probabilistic or deterministic) policy π that maximizes the value, i.e., the optimization problem the agent seeks to solve is:

$$\max_{\pi} V^\pi(s),$$

where the max is over all possibly non-stationary and randomized policies.

Next, let us look at the above problem formulation through several examples.

Example 5.1 (Robotics and control). *In a robotic manipulation or navigation task, the MDP is defined as follows. States $s \in \mathcal{S}$ represents continuous or discretized vectors representing joint angles, velocities, or the robot’s coordinates in 3D space. Actions $a \in \mathcal{A}$ could be torques applied to motors or velocity commands sent to the controller. Transitions $\Pr(s' | s, a)$ represent the physics of the environment (e.g., gravity, friction) which determines the next state, often with observational noise or uncertainty. Rewards $r(s, a)$ is a cost function based on distance to the target goal or a penalty for colliding with an obstacle.*

Example 5.2 (Conversational agent and RL reasoning). *In this domain, the MDP components are. States $s \in \mathcal{S}$: the transcript of the conversation and context history. Actions $a \in \mathcal{A}$: The set of possible responses or slot-filling queries. Transition model $\Pr(s' | s, a)$: The probability that a user responded with a transitioning to s' . Rewards $r(s, a)$: a binary outcome for task completion (e.g., successful booking).*

Example 5.3 (Strategic games). *For games like Go or Chess, states $\mathcal{S}$ is the set of all valid board configurations, actions $\mathcal{A}$ is the set of legal moves from a given state, transition model $\Pr(s' | s, a)$ is often deterministic for single-agent games, or represents the opponent’s response in multi-agent RL. The reward is 0 for intermediate steps, and +1 or −1 at the terminal state.*

Example 5.4 (RL Reasoning in LLMs). *In the context of LLMs trained with RL, the MDP is structured with states $s \in \mathcal{S}$ with the prompt plus the sequence of tokens generated so far. In reasoning models, this includes the “hidden” thoughts or reasoning chain. The actions $a \in \mathcal{A}$ is the next token to be sampled from the vocabulary. The transitions $\Pr(s' | s, a)$ to the next state s' is simply the previous sequence of tokens s concatenated with the sampled token a . The rewards $r(s, a)$ could be sparse, given only at the end of the sequence based on correctness (e.g., a math answer) or human preference, or dense, given for each step of the reasoning chain to encourage logical consistency.*

An important concept for studying RL problems is that of a stationary policy, which satisfies the following consistency conditions for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$, given V^π and Q^π :

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)] \\ Q^\pi(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim \Pr(\cdot|s, a)} [V^\pi(s')] \end{aligned}$$

Let us denote P^π as the transition matrix on state-action pairs induced by a stationary matrix π , specifically:

$$P_{(s,a),(s',a')}^\pi := \Pr(s'|s, a)\pi(a'|s'), \text{ for any } s' \in \mathcal{S}, a' \in \mathcal{A}$$

In particular, for deterministic policies π , we have

$$r_{(s,a),(s',a')}^\pi := \begin{cases} \Pr(s'|s, a) & \text{if } a' = \pi(s') \\ 0 & \text{if } a' \neq \pi(s') \end{cases}$$

Let V^π be a vector of length $|\mathcal{S}|$ and Q^π, r a vector of length $|\mathcal{S}| \times |\mathcal{A}|$. Let P be a matrix where $P_{(s,a),s'} = \Pr(s'|s, a)$. With this notation, we can verify that

$$Q^\pi = r + \gamma P V^\pi, \text{ or equivalently } Q^\pi = r + \gamma P^\pi Q^\pi,$$

leading to the solution that $Q^\pi = (\text{Id} - \gamma P^\pi)^{-1}r$. In particular, this step is based on the following expansion of $Q^\pi(s, a)$ on an arbitrary state-action pair (s, a) :

$$\begin{aligned} Q^\pi(s, a) &= r(s, a) + \gamma \sum_{(s', a') \in \mathcal{S} \times \mathcal{A}} \Pr(s' | s, a) \pi(a' | s') Q^\pi(s', a') & (Q^\pi = r + \gamma P^\pi Q^\pi) \\ &= r(s, a) + \gamma \sum_{s' \in \mathcal{S}} \Pr(s' | s, a) \underbrace{\sum_{a' \in \mathcal{A}} \pi(a' | s') Q^\pi(s', a')}_{V^\pi(s')} & (Q^\pi = r + \gamma P V^\pi) \end{aligned}$$

As a consequence of this equation, given a policy π , one can obtain the value function by solving a linear equation.

A remarkable and convenient property of MDPs is that there exists a stationary and deterministic policy that simultaneously maximizes $V^\pi(s)$ for all $s \in \mathcal{S}$. In particular, the Bellman optimality equations (Bellman, 1956) give a precise characterization of the optimal value function. In the next section, we will dive into this result.

5.1 Value iteration and policy iteration (Lecture 16)

Consider a finite-horizon and time-dependent MDP, $M = (\mathcal{S}, \mathcal{A}, \{P\}_h, \{r\}_h, H, \mu)$, specified with a state space $\mathcal{S}$, an action space $\mathcal{A}$, a time-dependent transition function $P_h : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$, a time-dependent reward function $r_h : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, an integer H which defines the horizon of the problem, and an initial state distribution $\mu \in \Delta(\mathcal{S})$ which specifies how the initial state s_0 is generated

Here, for a policy π , a state s , and $h \in \{0, \dots, H-1\}$, we define the value function $V_h^\pi : \mathcal{S} \rightarrow \mathbb{R}$ as

$$V_h^\pi(s) = \mathbb{E} \left[\sum_{t=h}^{H-1} r_t(s_t, a_t) | \pi, s_h = s \right]$$

Similarly, the state-action value function Q_h^π is defined as

$$Q_h^\pi(s, a) = \mathbb{E} \left[\sum_{t=h}^{H-1} r_t(s_t, a_t) | \pi, s_h = s, a \right]$$

Given a state a , the goal of the agent is to find a policy π that maximizes the value $\max_\pi V^\pi(s)$.

When the MDP is known, solving the optimal policy can be achieved in polynomial time using constrained optimization (Lan, 2023).

An important concept here for evaluating the value function is called the *Value Iteration*, an iterative method to compute the optimal value function V^* . Starting with an arbitrary V_0 , for each state $s \in \mathcal{S}$, we update:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} \Pr[s' | s, a] V_k(s') \right).$$

This algorithm continues until the value function converges, e.g., $\|V_{k+1} - V_k\|_\infty \leq \epsilon$ for some desired threshold ϵ . Once V^* is found, the optimal policy π^* is extracted via:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} \Pr[s' | s, a] V^*(s') \right).$$

Another fundamental concept, building on the Bellman's optimality condition, is called the *Policy Iteration*. In particular, policy iteration alternates between two steps until the policy π converges to π^* :

- *Policy evaluation*: Given the current policy π_k , compute the value function V^{π_k} by solving the linear system for all $s \in \mathcal{S}$

$$V^{\pi_k}(s) = r(s, \pi_k(s)) + \gamma \sum_{s' \in \mathcal{S}} \Pr[s' | s, \pi_k(s)] V^{\pi_k}(s')$$

- *Policy improvement*: Update the policy by being greedy with respect to V^{π_k}

$$\pi_{k+1}(s) = \arg \max_{a \in \mathcal{A}} \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} \Pr[s' | s, a] V^{\pi_k}(s') \right)$$

The algorithm terminates when $\pi_{k+1} = \pi_k$, guaranteeing that the optimal policy has been reached.

Example 5.5 (Strategic RLHF (Buening et al., 2025)). *Another interesting example with RLHF concerns data acquisition. Imagine that n agents or labelers are working to evaluate several language models. As in Example 5.4, a labeler is provided with a pair of (or several) trajectories in the form of prompt-response sequences, and is asked to provide their favorite trajectory, given an initial prompt.*

Under this scenario, a strategic agent may decide to misreport their true preference in the hope of manipulating the model behavior, which will be using the solicited preference trajectory to train the optimal policy. For example, if the aggregated optimal policy turns out to be highly undesirable, then an agent may choose to abstain by reverting their true preferences.

5.2 Sample complexity and online learning (Lecture 16)

For many sequential decision-making problems, we may not know the MDP a priori. Consider an online learning scenario. For $t = 1, \dots, T$, select decision $\pi^t \in \Pi := \{1, 2, \dots, A\}$, and observe reward $r^t \in \mathbb{R}$. The protocol proceeds in T rounds. At each round $t \in [T]$, the learning agent selects a discrete decision $\pi^t \in \Pi = \{1, \dots, K\}$ using historical data. Based on the decision π^t , the learner receives a random reward $r_a \in [-1, 1]$ from $R(a) \in \Delta([-1, 1])$, which has mean reward $\mathbb{E}_{r_a \sim R(a)}[r_a] = \mu_a$.

At every iteration t , the learner picks an arm $I_t \in [1, 2, \dots, K]$ according to a certain policy. An important concept is the cumulative regret with respect to the selected arms:

$$R_T = T \cdot \max_i \mu_i - \sum_{t=0}^{T-1} \mu_{I_t}.$$

Denote $a^* = \arg \min_i \mu_i$ as the optimal arm. Define gap $\Delta_a = \mu_{a^*} - \mu_a$ for any arm a .

Next, we shall describe the upper confidence bound (UCB) algorithm, which is a near-optimal procedure to identify the best arm Daniel et al., 2018. For simplicity, we allocate the first K rounds to pull each arm once. Then, for $t = 1 \rightarrow T - K$, execute arm

$$I_t = \arg \max_{i \in [K]} \left(\hat{\mu}_i^t + \sqrt{\frac{\log(TK/\delta)}{N_i^t}} \right), \text{ and observe } r_t := r_{I_t}, \quad (76)$$

where I_t is the index of the arm that is picked by the algorithm at iteration t , and the counts of each arm at each iteration t is

$$N_a^t = 1 + \sum_{i=1}^{t-1} \mathbf{1}\{I_i = a\}$$

In particular, the second term in equation (76) represents a confidence penalty inversely proportional to the empirical counts of pulling the i -th arm.

The upper confidence bound lemma (Foster and Rakhlin, 2023) states that with probability at least $1 - \delta$,

$$|\hat{\mu}_a^t - \mu_a| \leq 2\sqrt{\frac{\ln(TK/\delta)}{N_a^t}},$$

where $\hat{\mu}_a^t$ is the empirical mean for each arm. It can be shown that the UCB algorithm achieves a cumulative regret of at most $O(\sqrt{KT \ln(KT/\delta)} + K)$, which vanishes relative to the total reward as T increases.

The theory of RL is a fascinating topic that has been attracting interests in the ML community. For further reading, see, e.g., Foster and Rakhlin, 2023; Chi et al., 2025.

P.S.

- Q&A form: <https://forms.gle/SCjXUW6dkM8cDi4o9>.
- Latex source code: <https://github.com/hongyangsg/cs7140-advanced-ml>.

References

- Agarwal, A., Jiang, N., Kakade, S. M., and Sun, W. (2019). “Reinforcement learning: Theory and algorithms”. In: *CS Dept., UW Seattle, Seattle, WA, USA, Tech. Rep 32*, p. 96 (page 58).
- Alquier, P. (2021). “User-friendly introduction to PAC-Bayes bounds”. In: *arXiv preprint arXiv:2110.11216* (page 49).
- Arora, R., Arora, S., Bruna, J., Cohen, N., Du, S., Ge, R., Gunasekar, S., Jin, C., Lee, J., Ma, T., Neyshabur, B., and Song, Z. (2020). *Theory of deep learning* (page 36).
- Bach, F. (2024). *Learning theory from first principles*. MIT press (page 12).
- Bartlett, P. L., Long, P. M., Lugosi, G., and Tsigler, A. (2020). “Benign overfitting in linear regression”. In: *Proceedings of the National Academy of Sciences* 117.48, pp. 30063–30070 (pages 44, 45).
- Bellman, R. (1956). “Dynamic programming and Lagrange multipliers”. In: *Proceedings of the National Academy of Sciences* 42.10, pp. 767–769 (page 60).
- Buening, T. K., Gan, J., Mandal, D., and Kwiatkowska, M. (2025). “Strategyproof Reinforcement Learning from Human Feedback”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems* (page 61).
- Caruana, R. (1997). “Multitask learning”. In: *Machine learning* 28, pp. 41–75 (page 53).
- Catoni, O. (2007). “PAC-Bayesian supervised classification: the thermodynamics of statistical learning”. In: *arXiv preprint arXiv:0712.0248* (page 49).
- Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., and Wong, E. (2025). “Jailbreaking black box large language models in twenty queries”. In: *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, pp. 23–42 (page 47).
- Chi, Y., Chen, Y., and Wei, Y. (2025). “Statistical and algorithmic foundations of reinforcement learning”. In: *Tutorials in Operations Research: Advances in Analytics and Operations Research: Improving Decisions to Secure the Future*. INFORMS, pp. 104–144 (page 62).
- Daniel, J. R., Benjamin, V. R., Abbas, K., Ian, O., and Zheng, W. (2018). “A tutorial on thompson sampling”. In: *Foundations and Trends® in Machine Learning* 11.1, pp. 1–99 (page 61).
- Dhifallah, O. and Lu, Y. M. (2021). “Phase transitions in transfer learning for high-dimensional perceptrons”. In: *Entropy* 23.4, p. 400 (page 8).
- Du, S. S., Hu, W., Kakade, S. M., Lee, J. D., and Lei, Q. (2020). “Few-shot learning via learning the representation, provably”. In: *arXiv preprint arXiv:2002.09434* (page 57).

- Duchi, J. (2019). “Lecture notes for statistics 311/electrical engineering 377”. In: *URL: <http://web.stanford.edu/class/stats311/lecture-notes.pdf>* (page 9).
- Finn, C., Rajeswaran, A., Kakade, S., and Levine, S. (2019). “Online meta-learning”. In: *International conference on machine learning*. PMLR, pp. 1920–1930 (page 57).
- Foster, D. J. and Rakhlin, A. (2023). “Foundations of reinforcement learning and interactive decision making”. In: *arXiv preprint arXiv:2312.16730* (page 62).
- Garg, S., Tsipras, D., Liang, P. S., and Valiant, G. (2022). “What can transformers learn in-context? a case study of simple function classes”. In: *Advances in neural information processing systems* 35, pp. 30583–30598 (page 7).
- Ghadimi, S. and Lan, G. (2013). “Stochastic first-and zeroth-order methods for nonconvex stochastic programming”. In: *SIAM journal on optimization* 23.4, pp. 2341–2368 (page 39).
- Kouw, W. M. and Loog, M. (2018). “An introduction to domain adaptation and transfer learning”. In: *arXiv preprint arXiv:1812.11806* (page 7).
- Lan, G. (2023). “Policy mirror descent for reinforcement learning: Linear convergence, new sampling complexity, and generalized problem classes”. In: *Mathematical programming* 198.1, pp. 1059–1106 (page 60).
- Ledoux, M. and Talagrand, M. (2013). *Probability in Banach Spaces: isoperimetry and processes*. Springer Science & Business Media (page 21).
- Li, Y., Ma, T., and Zhang, H. (2018). “Algorithmic regularization in over-parameterized matrix sensing and neural networks with quadratic activations”. In: *Conference On Learning Theory*. PMLR, pp. 2–47 (pages 40, 41).
- McAllester, D. A. (1998). “Some pac-bayesian theorems”. In: *Proceedings of the eleventh annual conference on Computational learning theory*, pp. 230–234 (page 46).
- Mitzenmacher, M. and Upfal, E. (2017). *Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis*. Cambridge university press (pages 14, 23).
- Neyshabur, B., Bhojanapalli, S., McAllester, D., and Srebro, N. (2017). “Exploring generalization in deep learning”. In: *Advances in neural information processing systems* 30 (pages 28, 38).
- Ruder, S. (2017). “An Overview of Multi-Task Learning in Deep Neural Networks”. In: *arXiv preprint arXiv:1706.05098* (page 8).
- Shalev-Shwartz, S. and Ben-David, S. (2014). *Understanding machine learning: From theory to algorithms*. Cambridge university press (page 46).
- Srebro, N. and Shraibman, A. (2005). “Rank, trace-norm and max-norm”. In: *International conference on computational learning theory*. Springer, pp. 545–560 (page 46).
- Tulino, A. M. and Verdú, S. (2004). *Random matrix theory and wireless communications*. Now Publishers Inc (page 45).
- Valiant, L. G. (1984). “A theory of the learnable”. In: *Communications of the ACM* 27.11, pp. 1134–1142 (page 10).

- Wainwright, M. J. (2019). *High-dimensional statistics: A non-asymptotic viewpoint*. Vol. 48. Cambridge university press (pages 4, 12, 56).
- Wu, S., Zhang, H. R., and Ré, C. (2020). “Understanding and improving information transfer in multi-task learning”. In: *ICLR* (page 57).
- Zhang, H. R., Li, D., and Ju, H. (2024). “Noise stability optimization for finding flat minima: A hessian-based regularization approach”. In: *Transactions on Machine Learning Research* (page 52).
- Zhang, T. (2023). *Mathematical analysis of machine learning algorithms*. Cambridge University Press (page 12).